# Ch2. 개발환경 구축

파이썬 개발 환경 구축에 대한 챕터입니다. 이 책에서는 VS CODE 및 Anaconda 환경을 구축하는 방법에 대해 다룹니다.

# 2-1. 파이썬 설치(Anaconda)

파이썬 업무 자동화를 위해선 당연하게도 가장 먼저 파이썬을 설치해야합니다.

# 1. 파이썬 설치 방법 2가지

파이썬을 설치하는 방법은 크게 2가지가 있습니다.

1) 순수 파이썬 설치 (파이썬 공식 홈페이지)

2) 아나콘다 패키지 설치 (아나콘다 공식 홈페이지)

# 2. 아나콘다 패키지 추천 이유

저는 위의 두 가지 방법 중 아나콘다 패키지를 추천드립니다.

그 이유는 아래와 같습니다.

## 1) 데이터 과학 및 딥러닝 관련된 수 많은 라이브러리를 따로 설치할 필요 없음.

파이썬에서 어떠한 라이브러리를 사용하기 위해서는 터미널 창에서 pip install 명령어로 설치해주어야함.

이 때, 기존 라이브러리들과의 충돌 여부를 확인하지 않으므로, 새로운 라이브러리 설치가 잘 안되는 경우가 빈번함.

아나콘다는 데이터 과학 관련 라이브러리들이 이미 설치되어 있어, 위와 같은 작업이 줄어들 뿐만아니라, conda라는 패키지 관리자가 현재 환경과 충돌 여부를 설치 시에 확인하여, 문제가 적음.

## 2) 수행하는 프로젝트에 따라 가상 환경을 만들어 관리할 수 있음

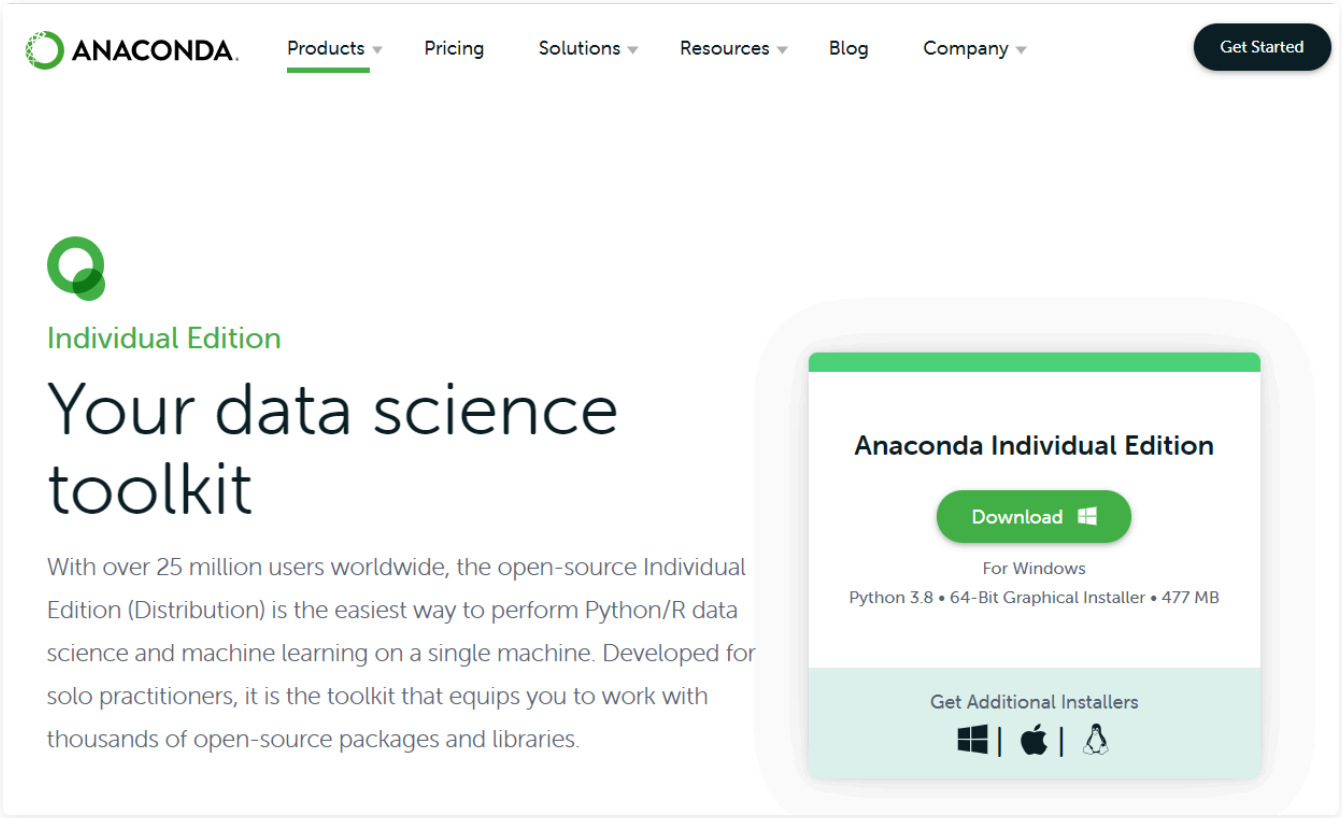
예를 들어, A 프로젝트에는 python2를 사용해야하고, B 프로젝트에는 python3을 사용해야 하는 경우가 있음.

이때 각 프로젝트에 맞는 가상의 환경을 만들어, 버전에 따른 버그를 방지할 수 있음.

# 3. 아나콘다 설치방법 (Windows OS)

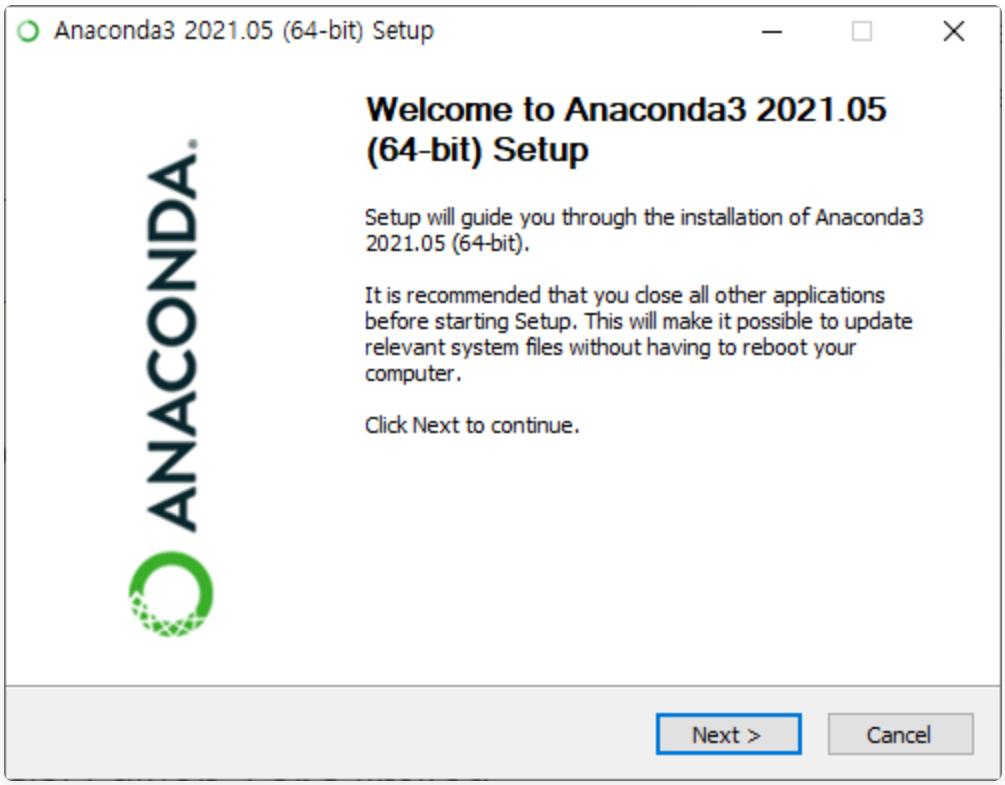
1. 아래의 링크 혹은 ANACONDA를 직접 검색하셔서 공식홈페이지에 접속한 후, 다운로드 버튼을 눌러줍니다.

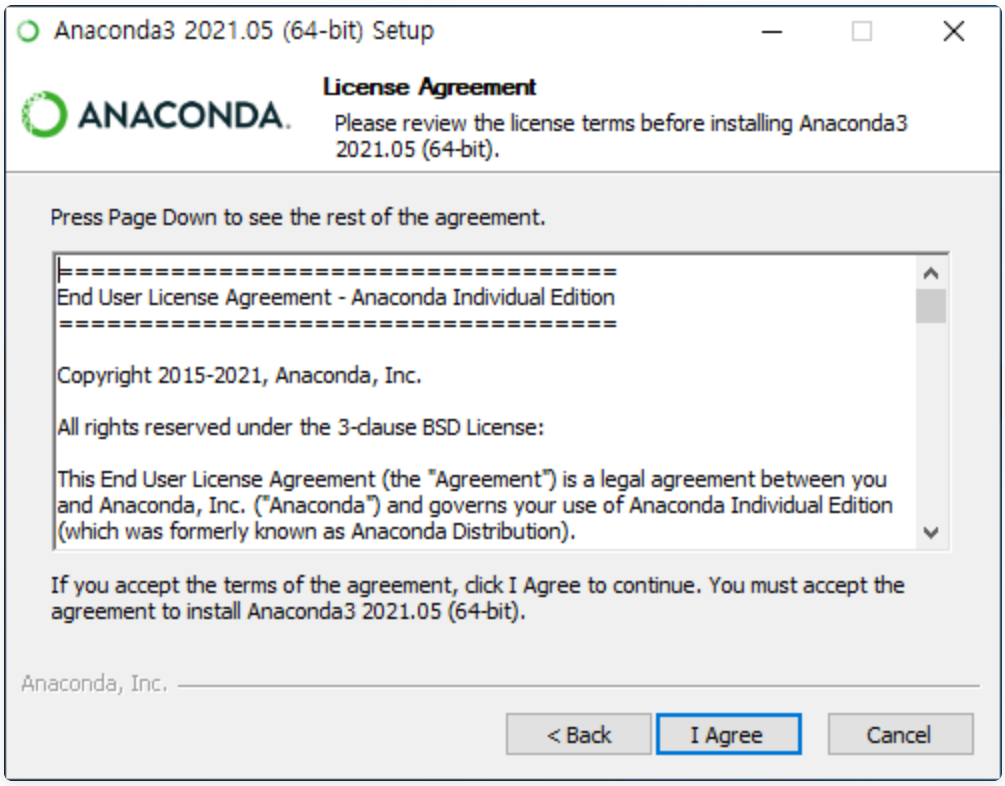
www.anaconda.com/products/individual

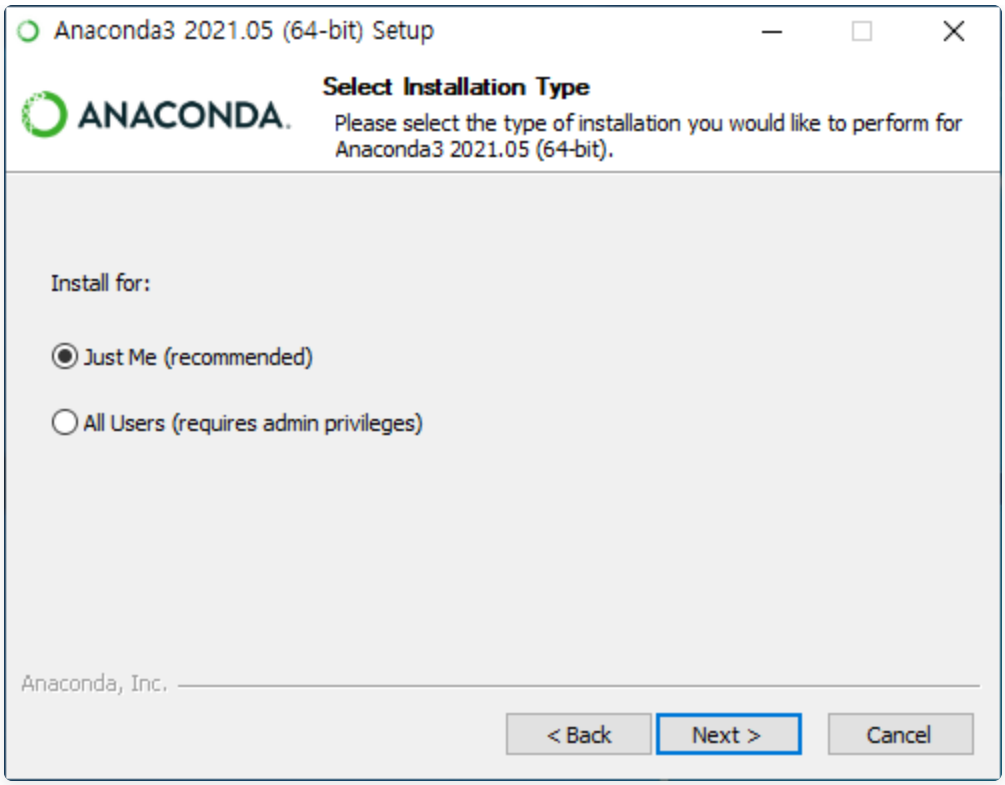



2. 다운로드가 완료되었으면, Setup파일을 실행시켜 줍니다.

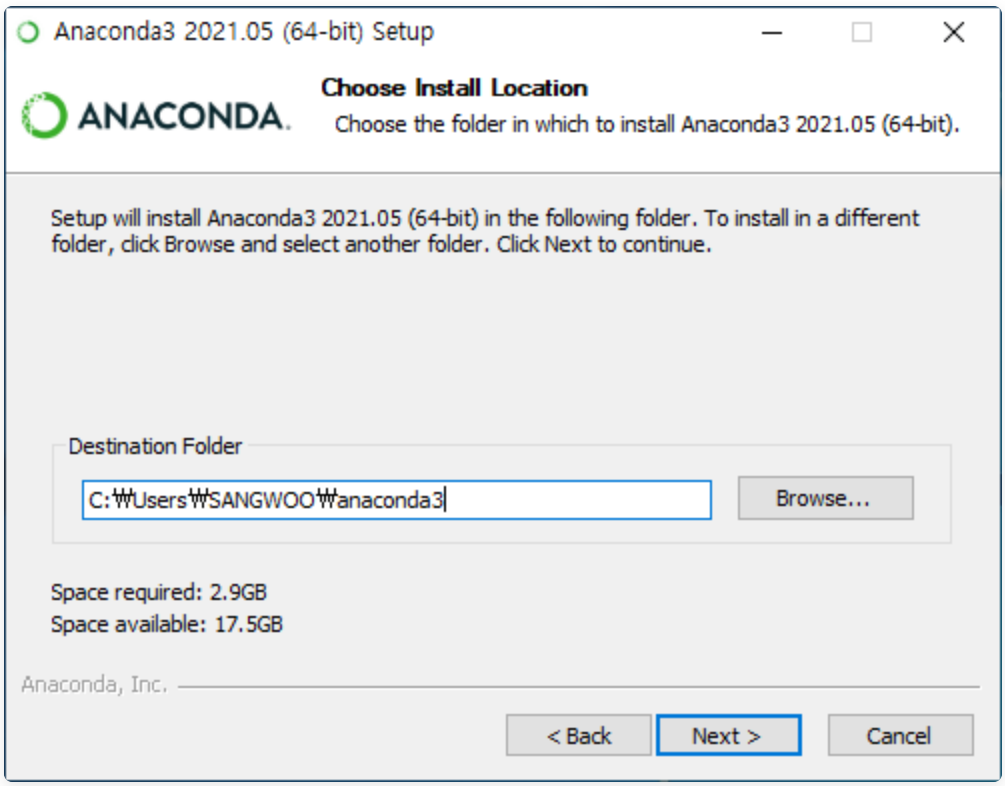
그 다음부터는 아래의 그림을 보시고 파란색으로 표시된 버튼을 차례대로 클릭해주세요.


Anaconda3 2021.05 (64-bit) Setup
Welcome to Anaconda3 2021.05 (64-bit) Setup
Setup will guide you through the installation of Anaconda3 2021.05 (64-bit).
It is recommended that you close all other applications before starting Setup. This will make it possible to update relevant system files without having to reboot your computer.
Click Next to continue.
Next >
Cancel

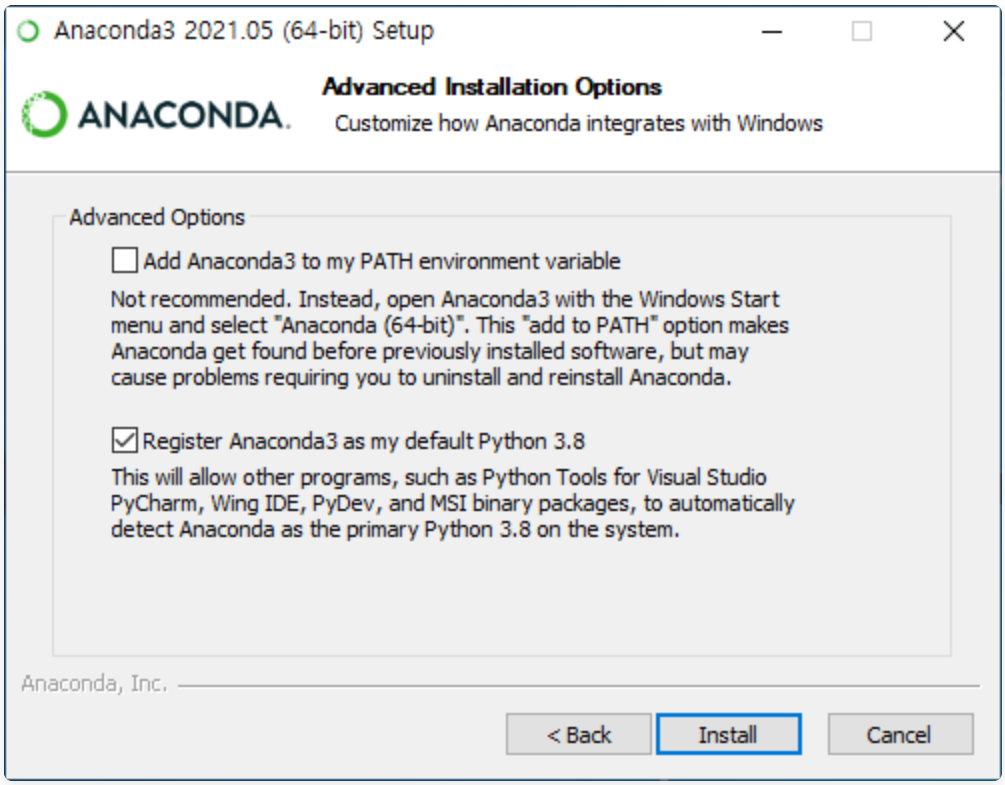

Anaconda3 2021.05 (64-bit) Setup
License Agreement
Please review the license terms before installing Anaconda3 2021.05 (64-bit).
Press Page Down to see the rest of the agreement.
End User License Agreement - Anaconda Individual Edition
Copyright 2015-2021, Anaconda, Inc.
All rights reserved under the 3-clause BSD License:
This End User License Agreement (the "Agreement") is a legal agreement between you and Anaconda, Inc. ("Anaconda") and governs your use of Anaconda Individual Edition (which was formerly known as Anaconda Distribution).
If you accept the terms of the agreement, click I Agree to continue. You must accept the agreement to install Anaconda3 2021.05 (64-bit).
Anaconda, Inc.
< Back
I Agree
Cancel




참고로 아래의 설치 경로를 설정할 때, 윈도우 사용자 이름이 영어가 아닌 한글로 설치되어 있을 때, 에러가 발생하며 설치가 되지 않습니다.

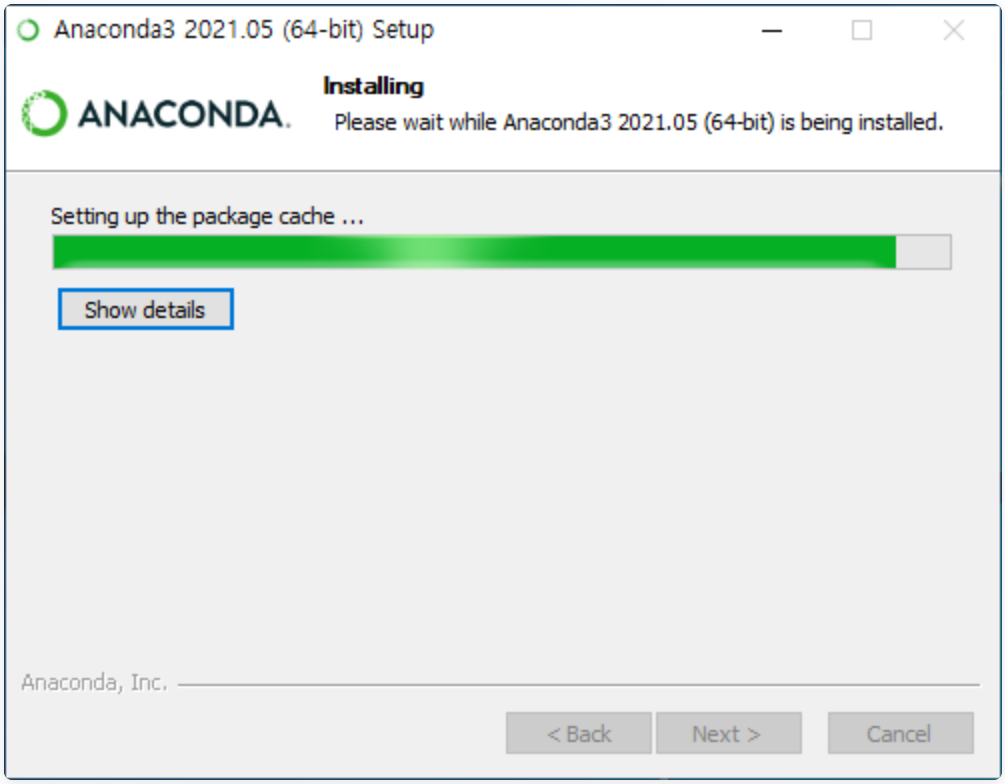



PATH 환경 변수 설정에 대한 내용인데 아나콘다에서 추천하는 두 번째 옵션으로 설정해줍니다. (기본으로 선택됨)

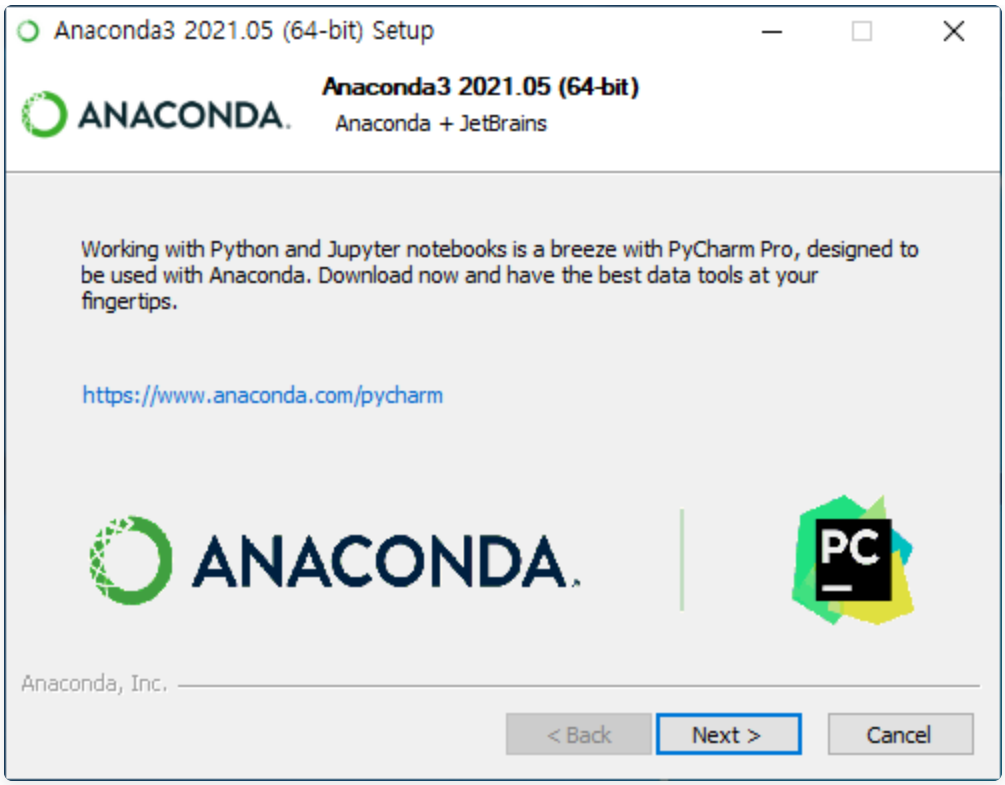



Install을 누른 후, 아래 그림의 상태에서 한참을 기다려야합니다. 컴퓨터 정지한거 아니니 걱정마시고 맘편히 기다려주세요.

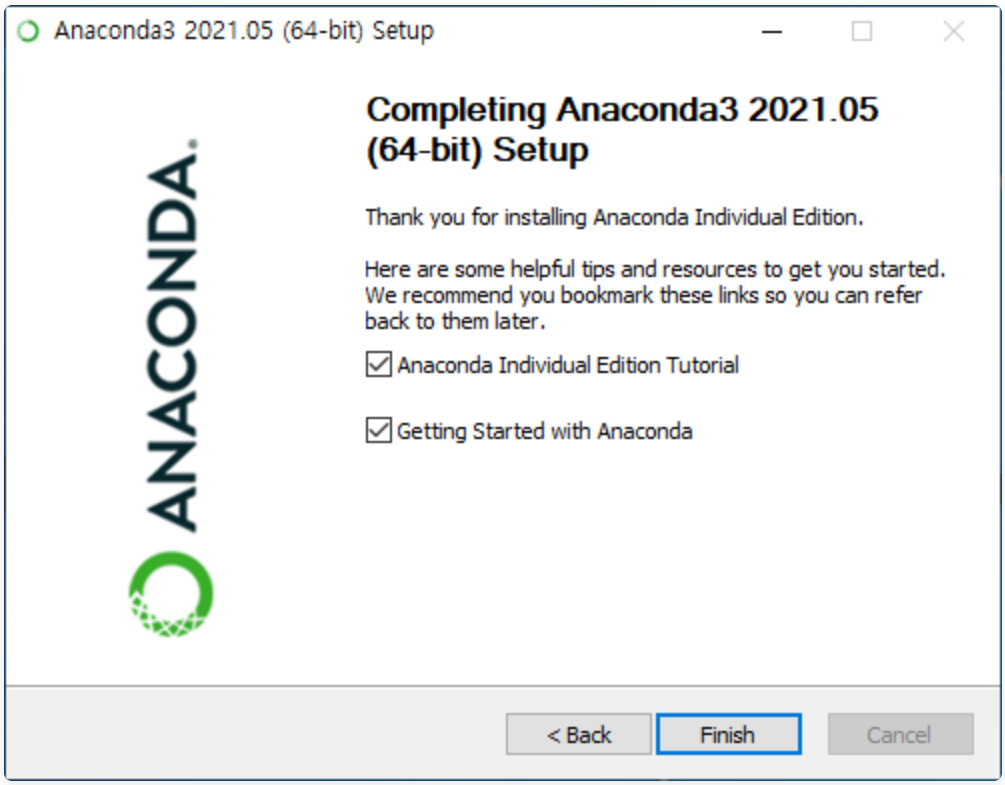
설치가 완료되면 Next 버튼을 클릭해주세요.




그냥 파이참 광고입니다. Next 눌러주세요.




설치가 완료되었습니다. Finish 버튼을 누르고 설치를 완료해줍니다.



# 2-2. IDE란?

오늘 배워볼 내용은 파이썬 개발을 도와주는 유용한 도구 IDE(Integrated Development Environment)입니다.

지난 글에서 파이썬과 아나콘다에 대해서 알아보았고, 아마도 여러분들은 더 유용한 아나콘다를 설치하셨을 것입니다.

파이썬을 설치하거나, 아나콘다를 설치했을 때에 기본적으로 제공되는 코드를 작성&실행할 수 있는 IDLE Shell이 있습니다.

하지만 마치 메모장에 코딩을 하는 느낌을 주기 때문에, 사용하기가 매우매우 불편합니다.

그래서!

우리는 지금부터 효율적인 파이썬 코딩을 위해서 IDE라 불리는 것에 대해서 알아보고 무엇을 사용할지 정해보는 시간을 가지겠습니다.

# 1. IDE란 무엇인가

IDE가 뭔지 알아보기 위해 네이버 지식백과를 검색해보았습니다.

> *IDE [Integrated Development Environment]*

효율적으로 소프트웨어를 개발하기 위한 통합개발환경 소프트웨어 어플리케이션 인터페이스이다.

코드 편집기, 디버거, 컴파일러, 인터프리터 등을 포함하고 개발자에게 제공한다.

위의 설명을 저의 얕은 지식으로 보충 설명 하자면 아래와 같습니다.

### 1) 코드 편집기

---

코드를 알록달록 예쁘게, 자동 완성, 자동 들여쓰기, 맞춤법 검사 등의 기능으로 코드 편집을 쉽게 해주는 역할

### 2) 디버거

---

코딩을 할 때, 코드가 잘 돌아가는지, 변수에는 어떤 값이 들어가 있는지 모니터링 해주는 역할

### 3) 컴파일러

---

작성된 코드 전체를 컴퓨터가 알아먹기 쉬운 언어(기계어)로 번역해서 작동시켜주는 역할

### 4) 인터프리터

---

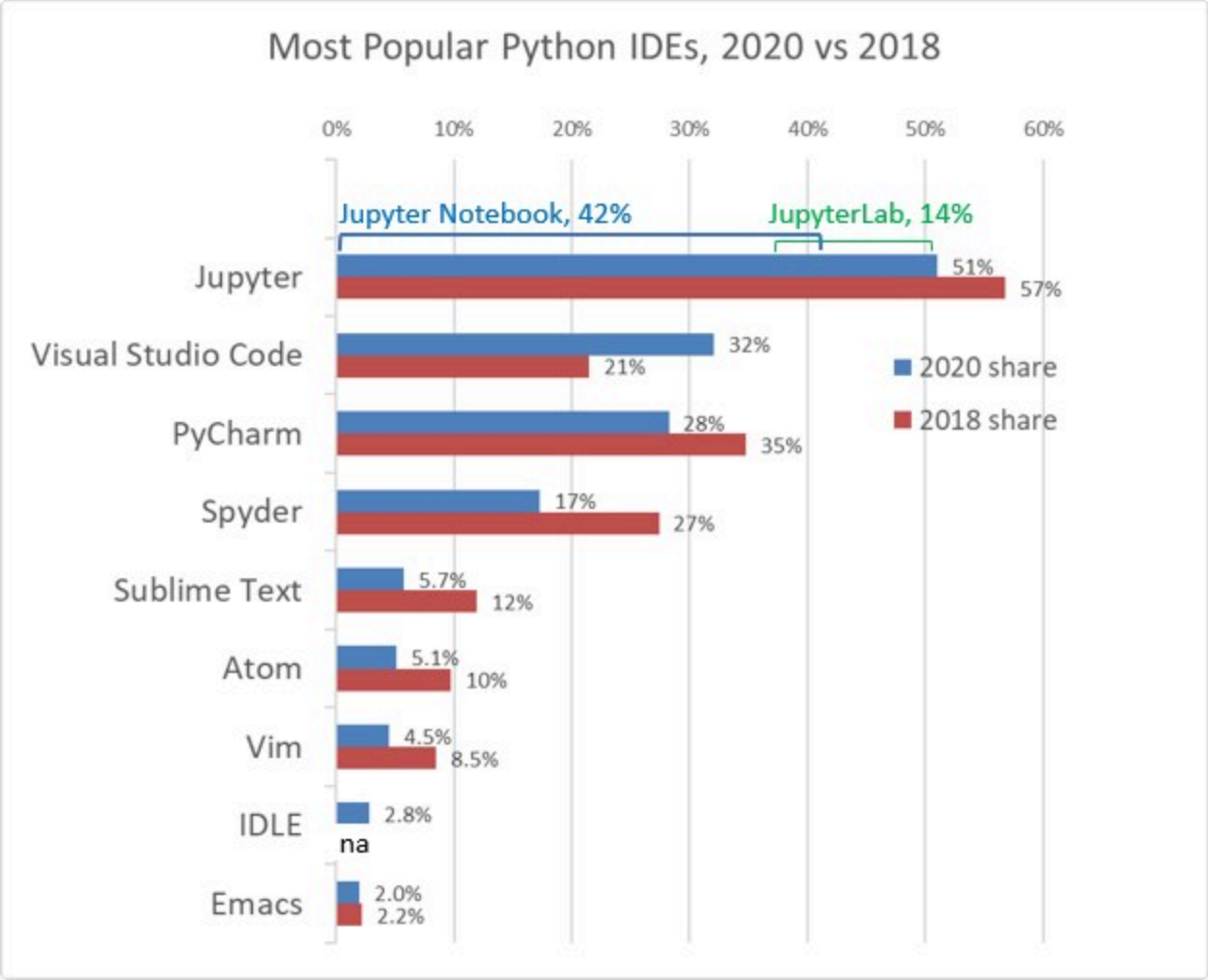
코드를 한 줄 한 줄 실행시켜주는 역할

# 2. 파이썬 IDE 사용 순위

---

그렇다면 현재 파이썬 유저들이 가장 많이 사용하고 있는 IDE는 무엇일까요?

한 설문 조사에서 2500개 이상의 응답 (최대 3개 중복 선택)으로 통계를 내보았을 때의 결과는 아래와 같다고 합니다.




**1위는 아나콘다에서 기본으로 제공하는 웹기반 IDE인 Jupyter Notebook이 차지했네요.**

**그리고 2위는 최근 많은 분들이 사용하고 계시는 VS Code가, 3위는 파이썬 전용 IDE인 PyCharm이 차지했습니다.**

# 3. 파이썬 IDE Top3 장단점

각 IDE들의 장단점은 아래와 같습니다.

## 1) Jupyter Notebook

아래의 장점으로 인해 주로 데이터 사이언스에 많이 활용됨

**장점**

1. 코드를 블록 단위로 쪼개어 작성, 실행하여 결과를 바로바로 확인 가능
2. 시각화 라이브러리를 이용해 표와 그래프를 즉시 확인 가능

**단점**

1. 디버깅 기능이 없음
2. 웹 기반이므로 항상 모든 코드를 처음부터 실행해야 함
3. ipynb라는 전용 확장자를 사용해서 다른 프로그램에서 사용하기가 어려움 (VS Code에서는 되는 것 확인)

## 2) Visual Studio Code

마이크로 소프트에서 만든 코드 편집기 (태생이 IDE가 아니었음..?!)

확장 기능의 수도 굉장히 많고, 이를 통해 다양한 종류의 언어를 지원하여 사실상 준 IDE급의 위치에 도달

**장점**

1. 완전 무료
2. 가볍고 시작 속도가 빠름
3. 확장 프로그램 설치를 통해 다양한 언어 코딩 가능 (그 중 하나가 파이썬)
4. 한국어 패치 가능

5. 코드 편집 UI가 심플하고 알아보기 쉬움

**단점**

PyCharm에 비해 파이썬 코딩 시, 기능이 조금 모자란다고 하는데 저는 아직까지 잘 모르겠습니다.

## 3) PyCharm

**장점**

1. 코드별로 파이썬 버전 선택 가능
2. GUI로 편리하게 라이브러리(패키지) 설치 가능
3. 파이썬 코딩에 최적화

**단점**

1. 오직 Python 전용 IDE
2. 프로그램이 무거워서 시작할 때 오래걸린다고 함
3. 무료 버전인 Community와 유료 버전인 Professional으로 나뉨

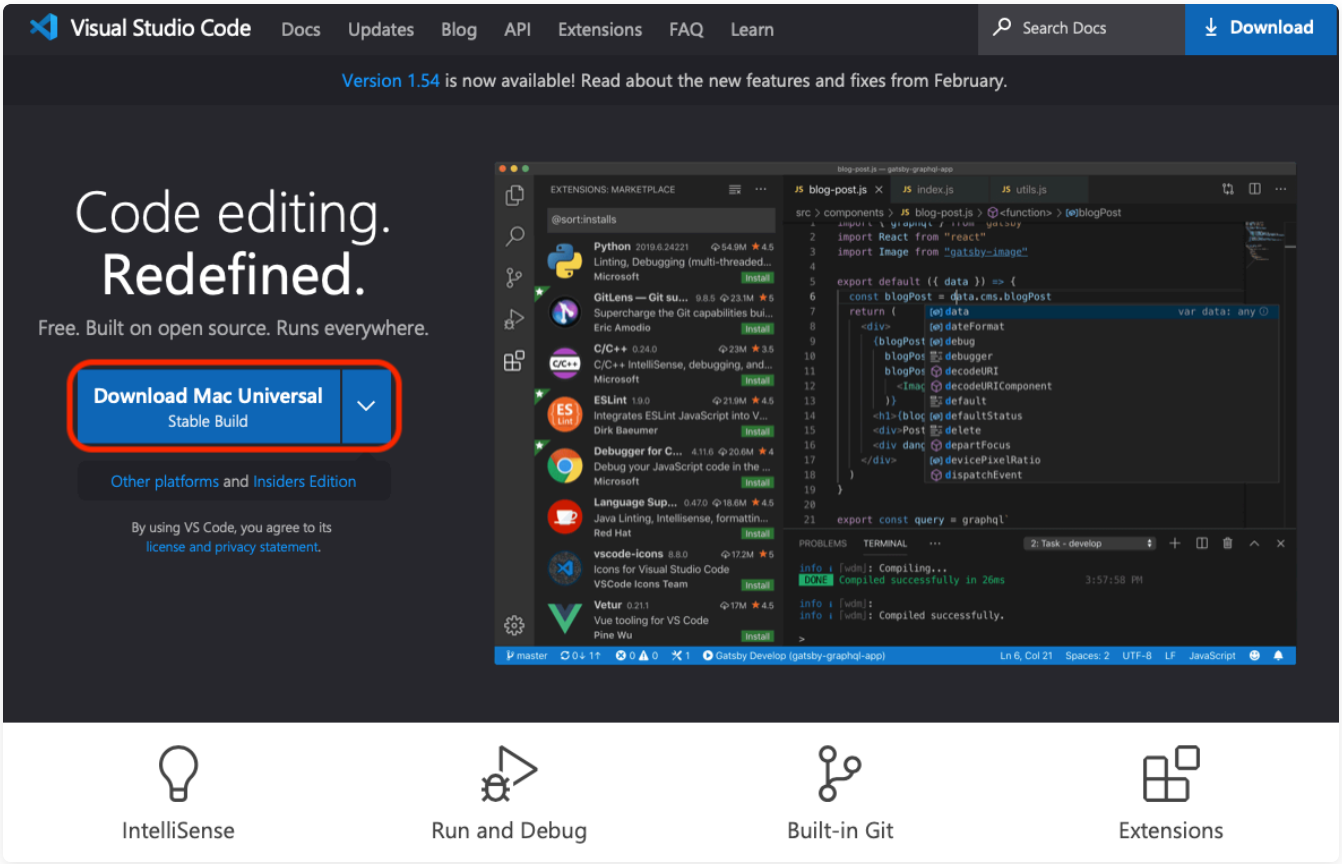
<참고자료>

zephyrnet.com/ko/here-are-the-most-popular-python-ides-editors/

# 2-3. VS CODE 설치

## 1. 홈페이지 접속 및 다운로드

code.visualstudio.com




## 2. 설치

### 1) Mac OS

단 한줄로 설명이 가능합니다.

다운받고 압축을 풀고 실행을 하면 바로 실행이 됩니다.

### 2) Windows OS

마찬가지로 매우 쉽습니다.

다운로드 파일 실행 후, 동의함, 다음, 다음, 다음, 설치, 마침

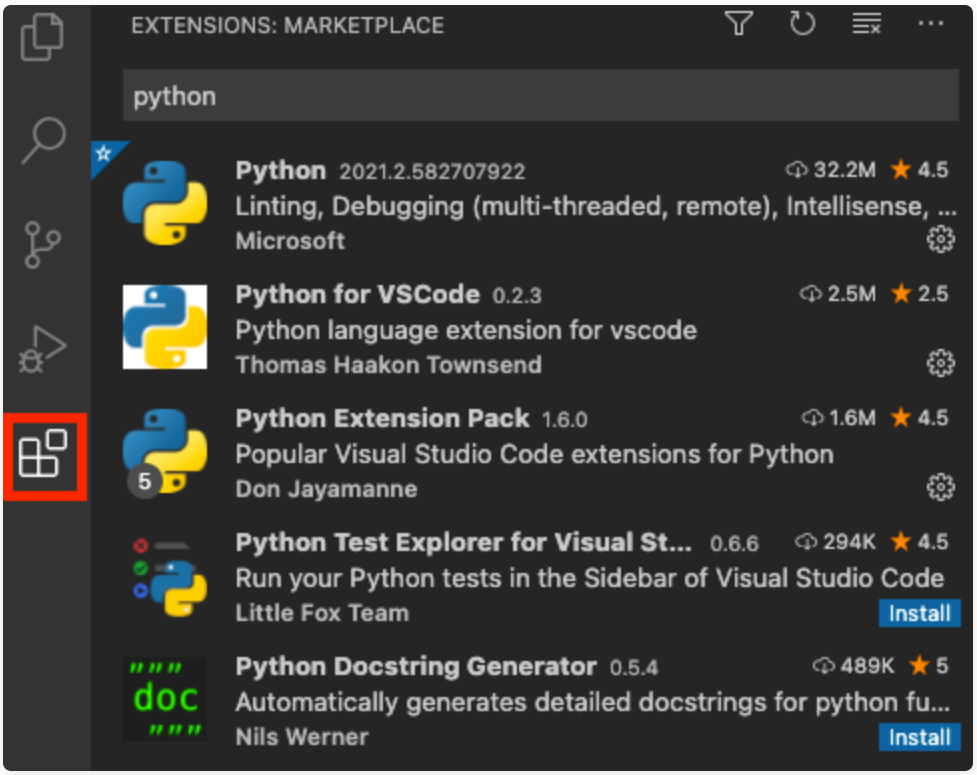
# 3. 필수 익스텐션 설치

VS code에서 파이썬을 사용하기 위해서는 몇 가지 EXTENSIONS을 설치해주어야 합니다.

VScode에서 파이썬을 사용하기 위해서 꼭 설치가 필요한 익스텐션들은 아래와 같습니다.

(저는 이미 설치가 되어있어서 안뜨지만 여러분들은 아래의 다른 익스텐션 처럼 Install 버튼이 있을거에요!)

- Python
- ~~Python for VSCode~~ (2022.08 기준 해당 익스텐션 지원X)
- Python Extension Pack

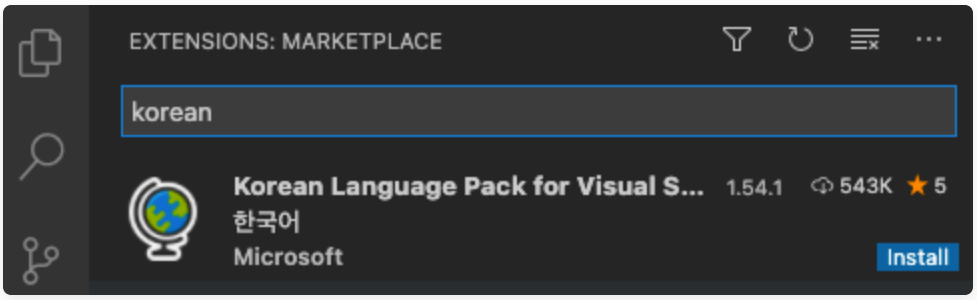


# 4. 추천 익스텐션 설치

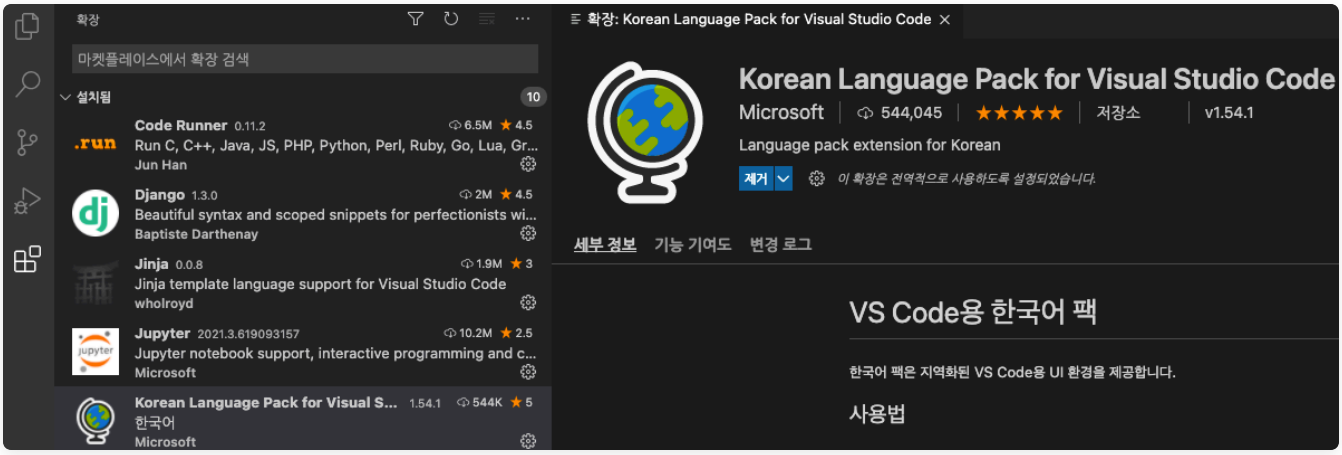
## 1) Korean Language Pack for Visual Studio

위와 같은 EXTENSIONS 탭에서 korean을 검색하면 아래와 같은 익스텐션이 나옵니다.

Install 버튼을 누르면 화면의 오른쪽 아래에 Restart라고 나오는데 누르시면 VS Code가 재시작 되면서 친숙한 한글이 펼쳐집니다.



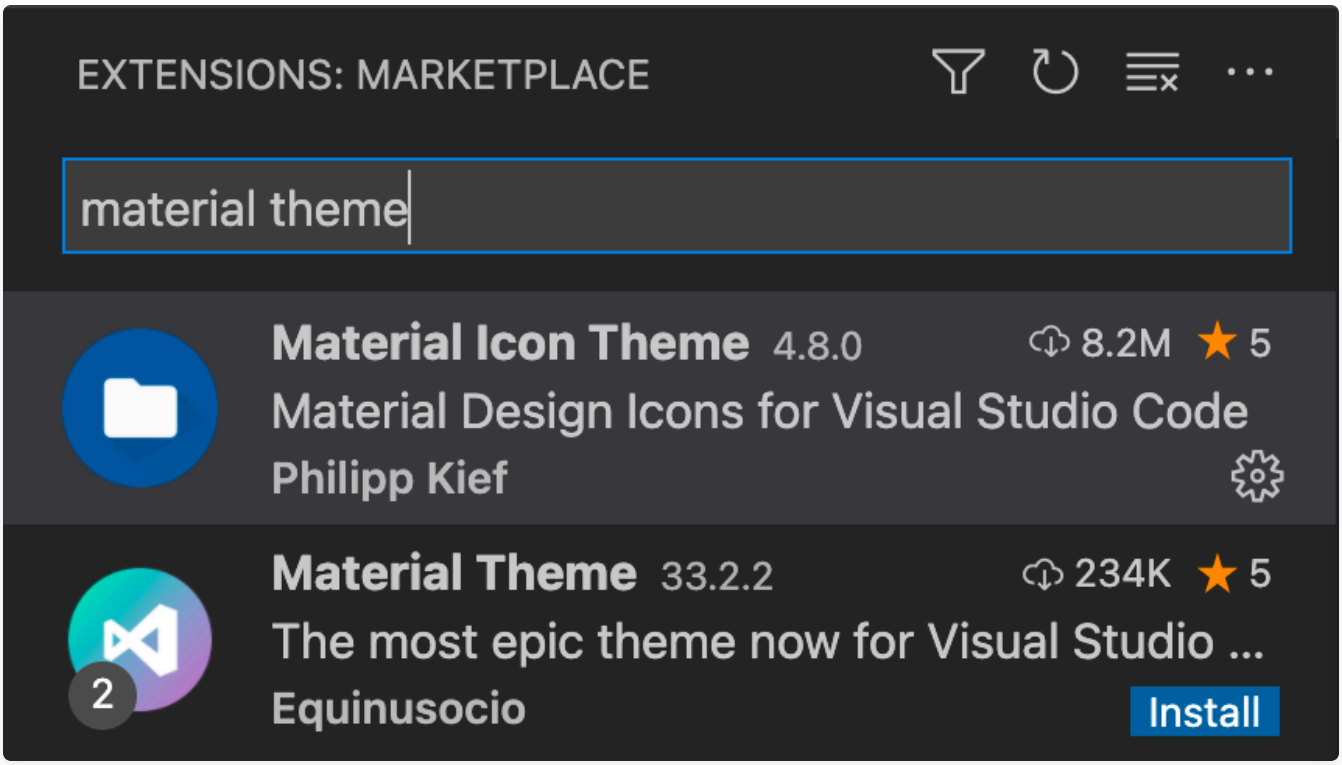

혹시나 설치하셨는데 '나는 코딩하면서 영어 공부도 할거야!' 하시는 분은 다시 해당 익스텐션을 클릭하면 제거 버튼이 나오니 참고하세요~




## 2) Material Theme

다음으로 소개드릴 Extension은 Material Theme입니다. VScode 좌측의 Extensions 탭을 클릭하신 후, 검색창에 아래와 같이 material theme이라고 검색해줍니다.

그러면 아래와 같이 두 가지 Extension이 나타날텐데 그냥 둘 다 Install 해주세요. 어차피 이어서 설치할테니까요!

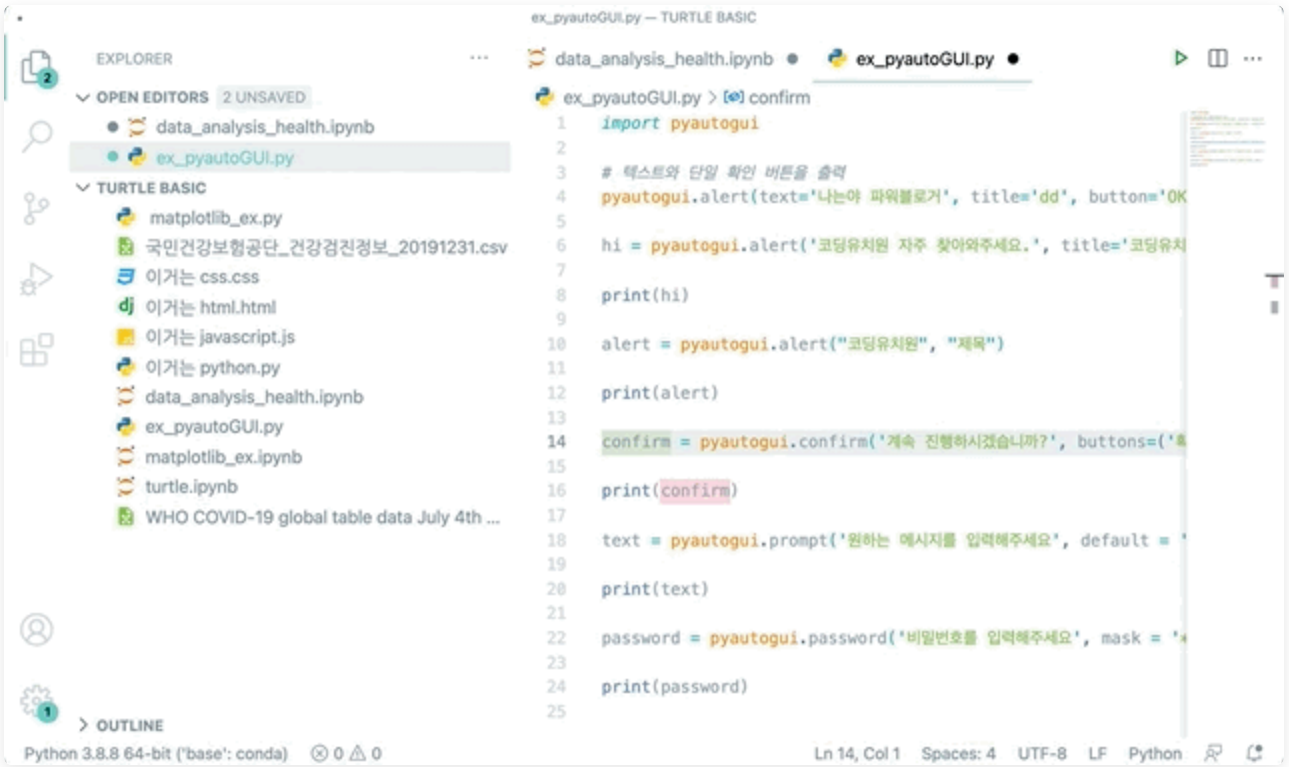



설치가 완료되었다면 사용방법은 간단합니다.

1) 좌측하단의 설정 아이콘을 클릭 후, Color Theme을 선택

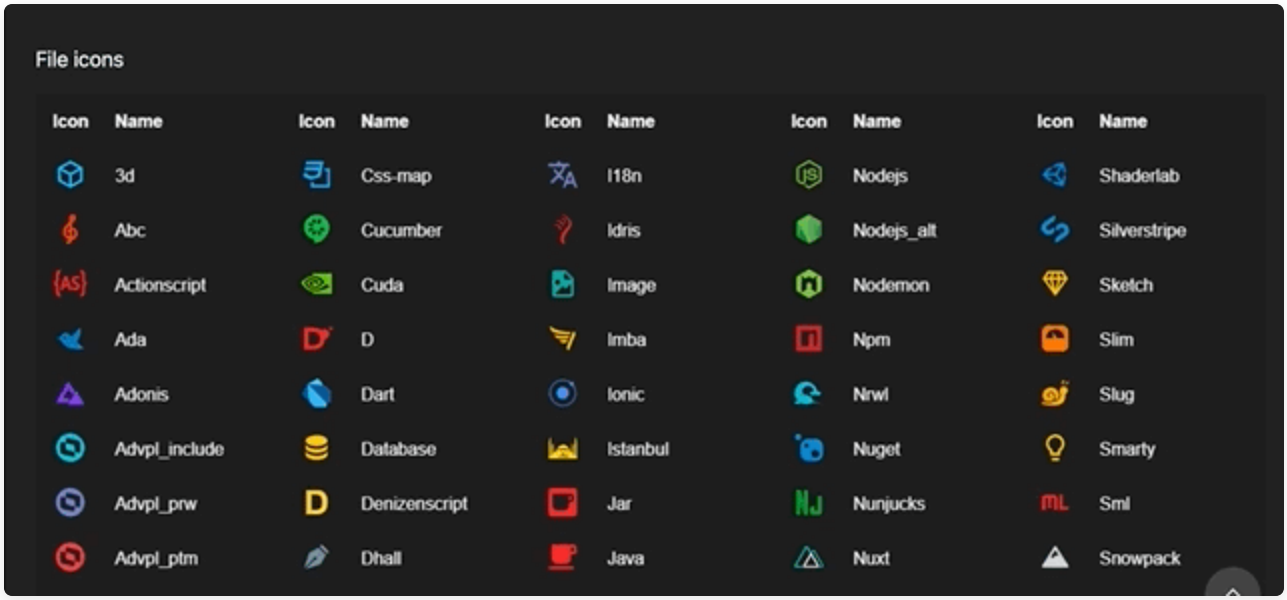
2) 상단에 출력되는 여러 테마들을 클릭해보면서 마음에 드는 테마를 선택해주면 끝!

참 쉽죠?



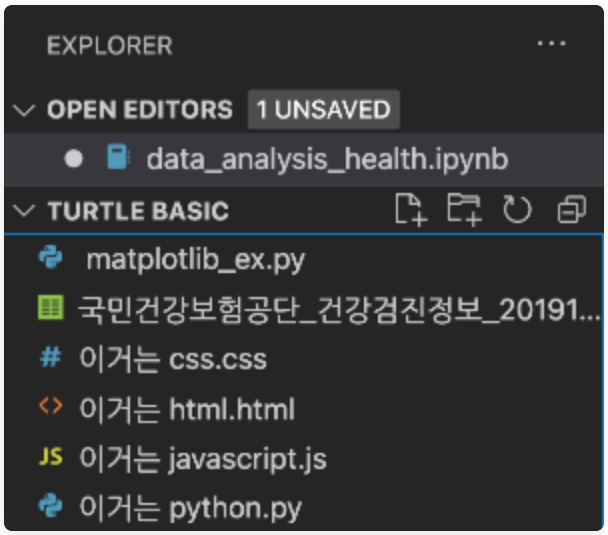
## 3) Material Icon Theme

세번째로 소개드릴 Extension은 Material Icon Theme입니다. 이 Extension은 코딩에 실질적으로 도움이 되지는 않지만 그냥 탐색기에 보이는 폴더와 파일이 예뻐보여서 사용합니다.

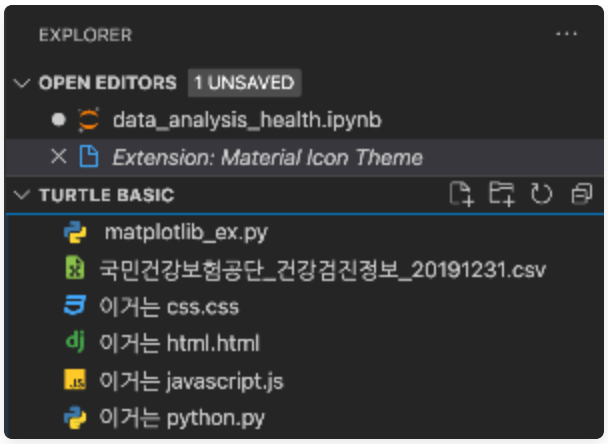



다양한 Icon Theme

설치 전과 후의 비교 사진입니다. 보면 아시겠지만 적용 전에도 아이콘이 없는 것은 아니랍니다.




적용 전

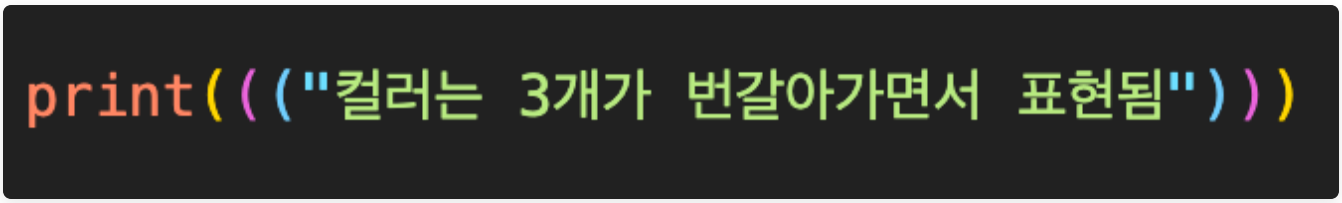



적용 후

## 4) Bracket Pair Colorizer 2

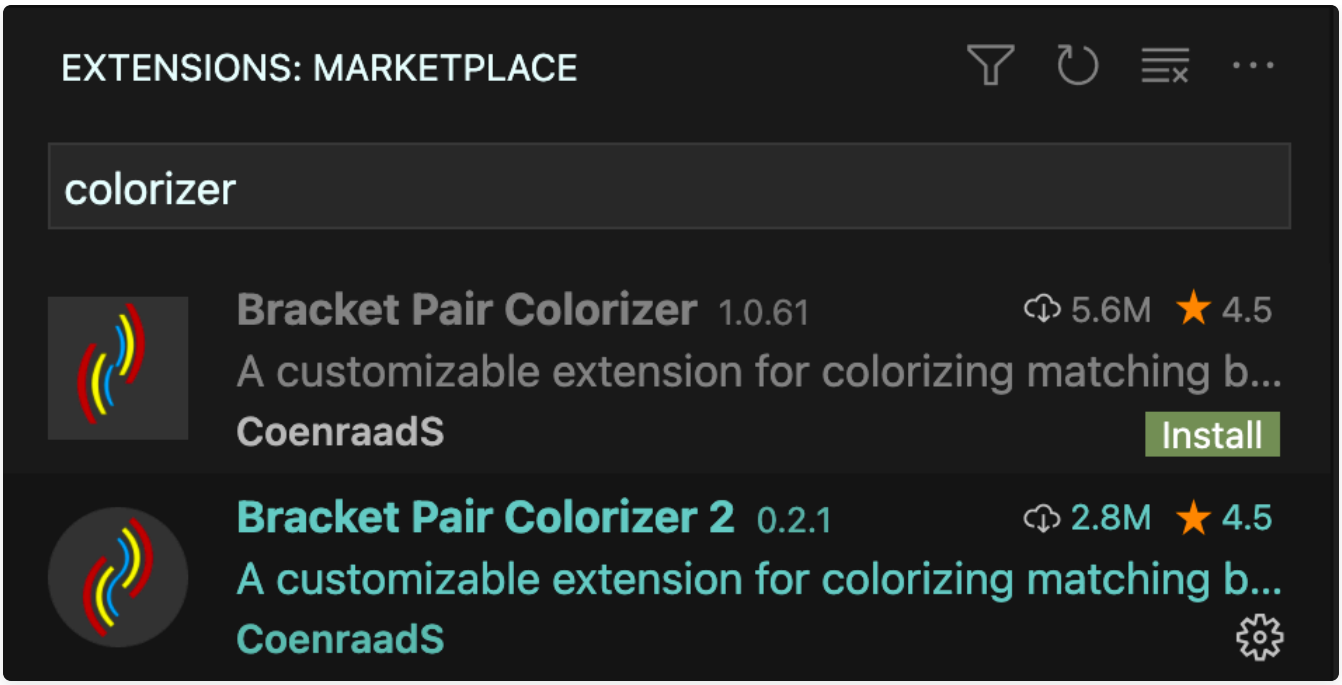
네번째로 알려드릴 Extension은 Bracket Pari Colorizer입니다. 이 Extension은 코딩을 하면은 무조건 사용하게 되는 대괄호[ ], 중괄호{ }, 소괄호( )에 색깔을 입혀주는 역할을 합니다.

괄호가 많아지게 되면 같은 괄호끼리는 어디가 열렸고 닫혔는지 헷갈릴 수 있는데, 그럴 때 도움이 될 것 같네요.




설치 시 바로 적용됨

마찬가지로 설치방법은 colorizer를 검색한 후, Install을 눌러주시면 됩니다. 여기서 주의하셔야 될 점은 버전 1과 버전 2가 공존하는데, 버전 2를 설치해주시는 걸 추천드립니다. 이유는 아래에 설명해두었습니다.




버전 2의 F.A.Q를 확인해보니 아래와 같이 설명해 놓았습니다.

# Bracket Pair Colorizer 2

This extension allows matching brackets to be identified with colours. The user can define which tokens to match, and which colours to use.

Screenshot:

```
22      public updateDocument(document: TextDocument) {
23          const documentDecoration = this.getDocumentDecorations(document);
24          if (documentDecoration) {
25              documentDecoration.triggerUpdateDecorations();
26          }
27      }
```

F.A.Q.

- Differences between v1 and v2?
  - v2 Uses the same bracket parsing engine as VSCode, greatly increasing speed and accuracy. A new version was released because settings were cleaned up, breaking backwards compatibility.

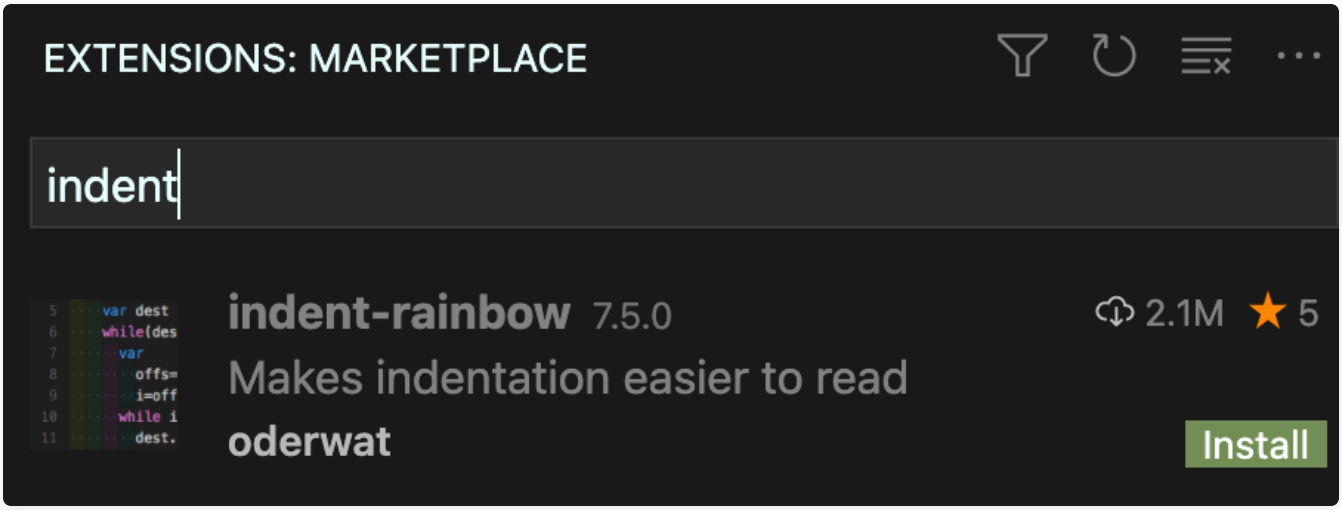
요약하자면,

1) 속도와 정확도를 크게 높임

2) 하위 호환성이 손상되어 새 버전이 출시하게 됨

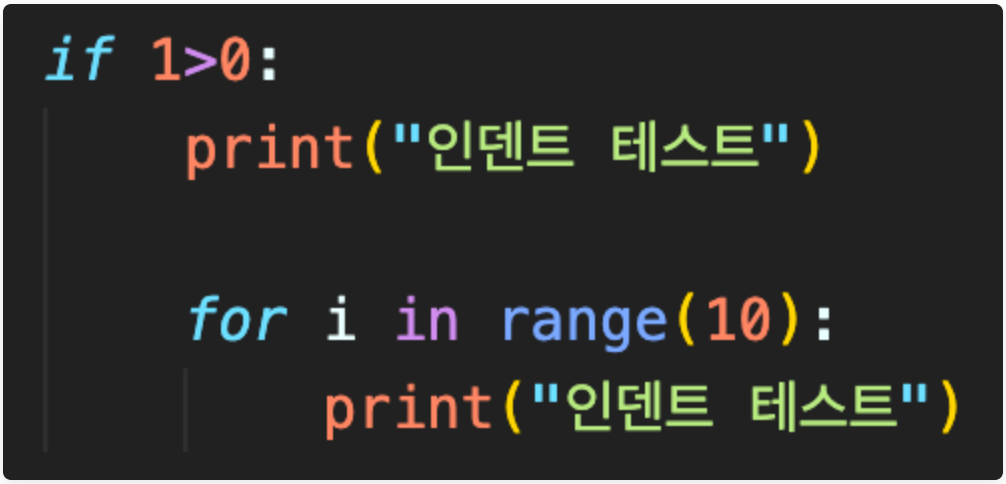
## 5) indent-rainbow

오늘 마지막으로 소개드릴 Extension은 indent-rainbow 입니다. 이름에서 딱 느낌이 오시겠지만 indent(들여쓰기)를 할 때, 무지개 색으로 구분을 해주는 Extension입니다.




indent-rainbow 설치

설치 전과 후의 이미지는 아래와 같습니다. indent 마다 노란색, 초록색, 빨간색, 파란색이 반복된답니다.



```
if 1 > 0:
    print("인덴트 테스트")

    for i in range(10):
        print(
            "인덴트 테스트")
        if 1 > 0:
            print("인덴트 테스트")
            if 1 > 0:
                print("인덴트 테스트")
                if 1 > 0:
                    print("인덴트 테스트")
```

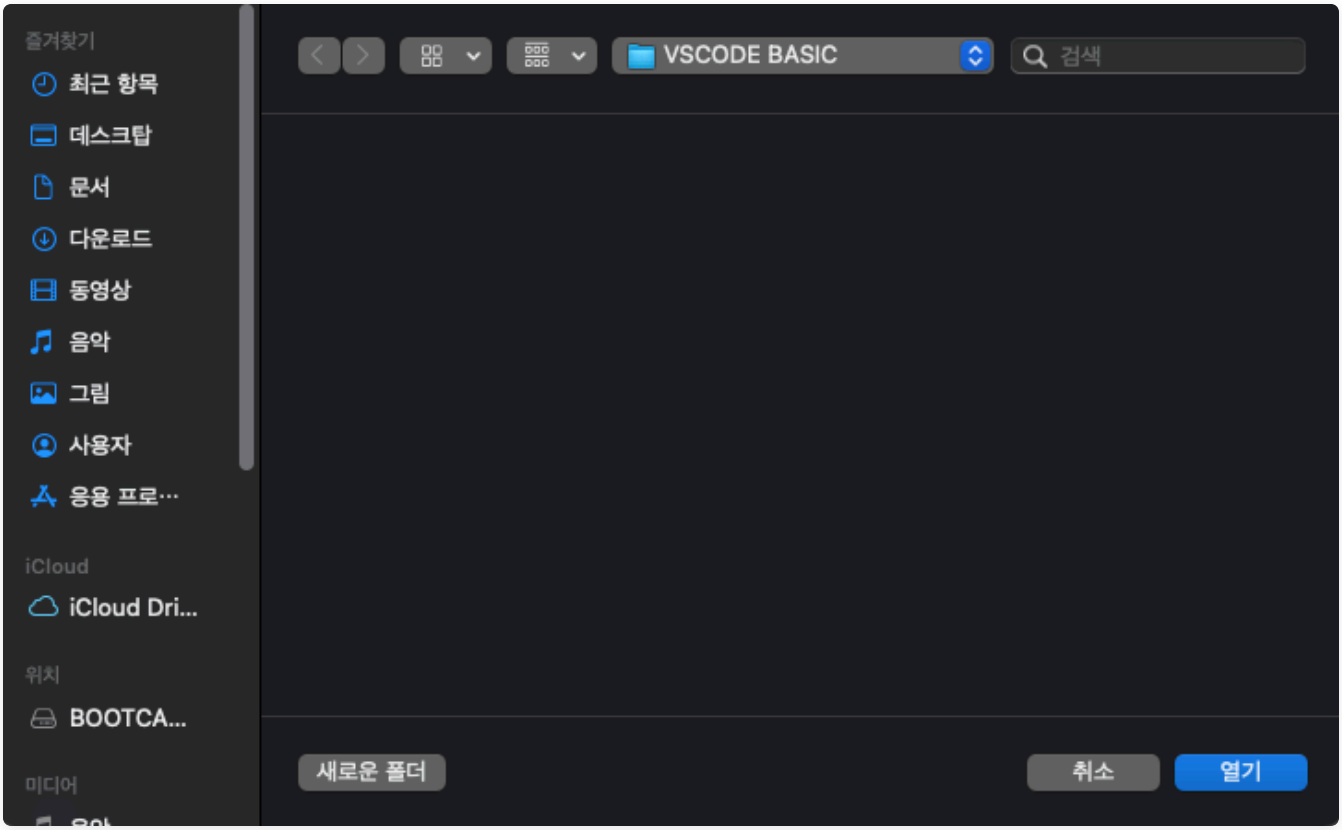
# 2-4. VS CODE 사용법 (기초)

지난 시간에 VS code를 설치하고, 파이썬 코딩을 위해 파이썬 관련 Extension 몇가지를 설치해보았습니다.

이번 시간에는 VS cdoe를 본격적으로 사용하기 위해서, 주요 기능들을 알아보겠습니다.

추가로 조금 설정할 것들이 있어서 그 부분도 간략히 설명하도록 하겠습니다.

# 1. 폴더 생성 & Python 파일 만들기




가장 처음할 것은 파이썬 파일이나 기타 파일들을 전부 넣어두는 하나의 프로젝트용 폴더를 만드는 것입니다.

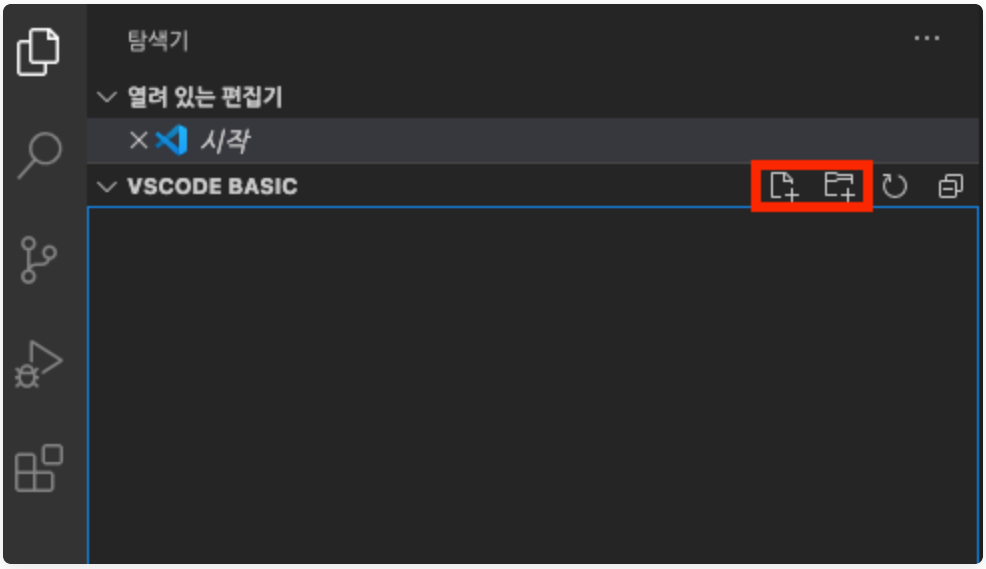
위치는 원하시는 어느 곳이든 상관없습니다. 이름은 제가 테스트해보니 한글로 만드셔도 작동하지만 웬만하면 영어로 해주세요.

보통 외국 프로그램들은 한글이랑 안 친해서 오류가 생기는 경우가 많거든요..

그리고 폴더를 생성하지 않고 그냥 바탕화면이나 기존 폴더를 사용해도 되지만 추천드리지 않아요.

나중에 파일을 찾을 때도 힘들고, 프로젝트를 하면서도 직관성이 떨어지니까요!

폴더를 만드셨으면 메뉴 텝에서 파일-열기를 선택하셔서 생성하신 폴더를 열어주세요.

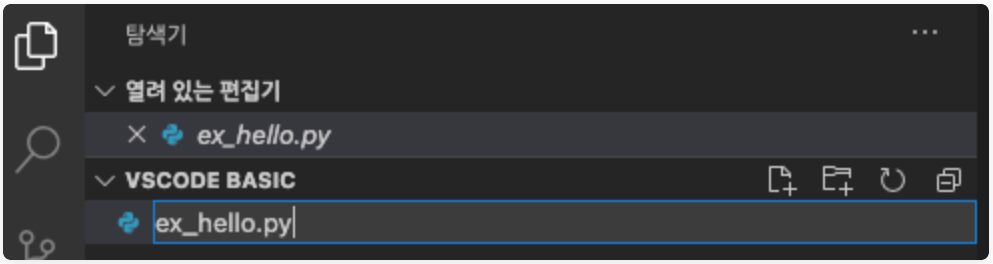



그러면 위의 그림처럼 화면이 나올텐데요. 좌측 상단을 보시면 +표시가 있는 아이콘이 2개 보이실 거에요.

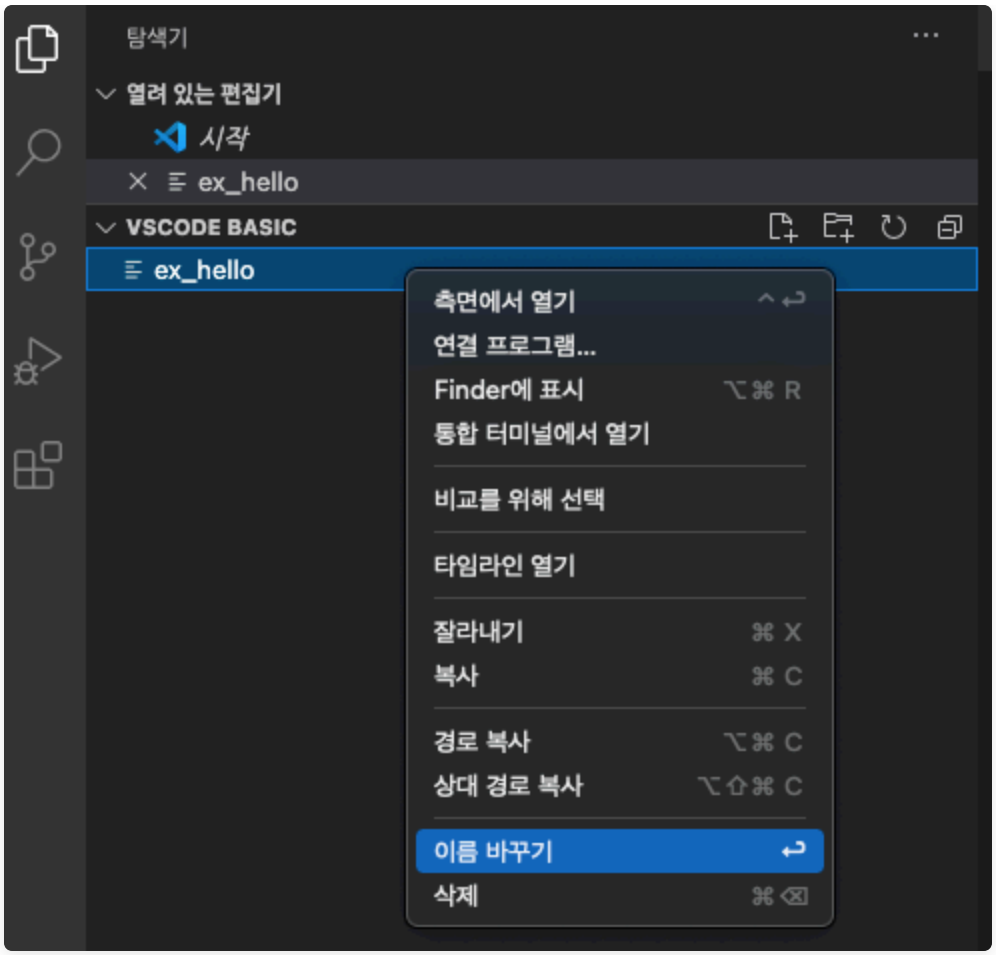
왼쪽이 파일을 생성하는 아이콘이고, 오른쪽이 폴더를 생성하는 아이콘입니다. (물론 우리가 방금 만든 프로젝트 폴더 안에요)

파이썬 파일(.py)을 만들기 위해서는 파일 생성 아이콘을 클릭 후, **파일 이름.py**로 이름을 지어주세요.

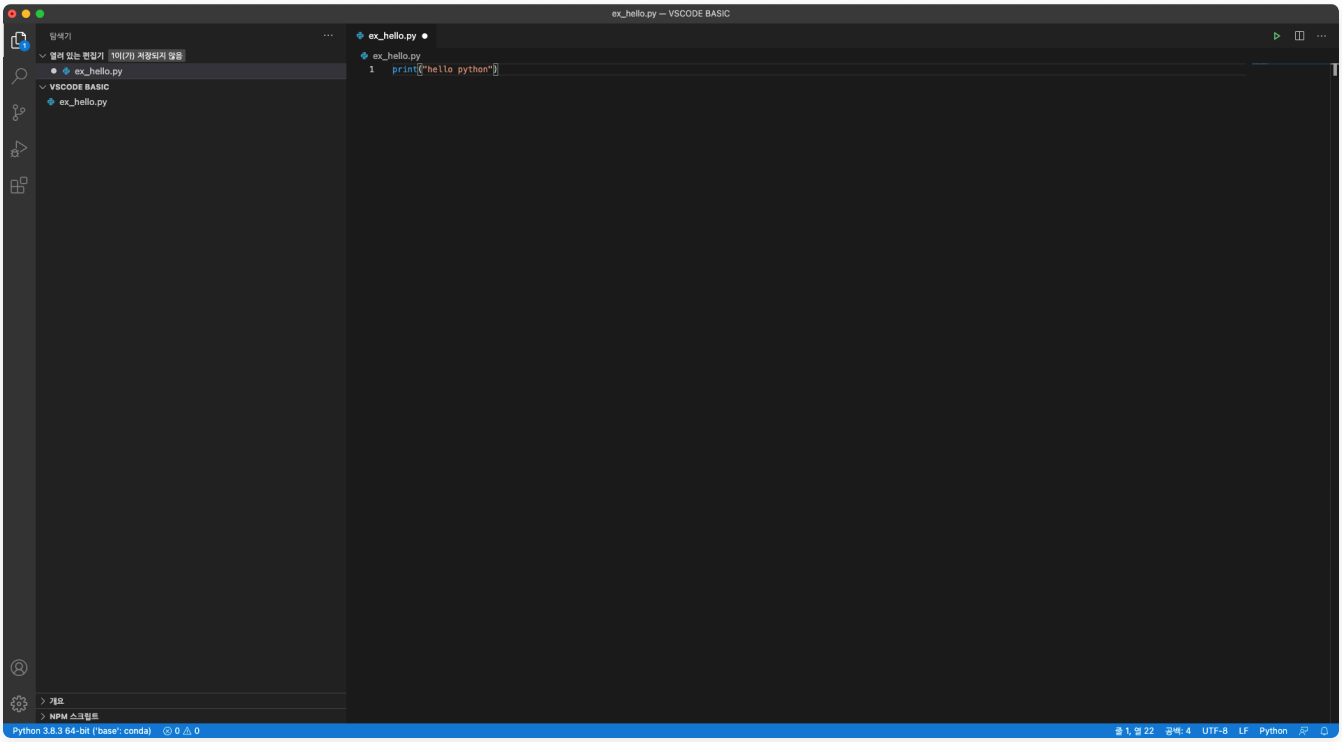
저는 ex_hello.py로 파이썬 파일을 생성했습니다.




만약에 파일 이름에 .py를 붙이지 않으셨다면 당황하지 마시고, 만드신 파일을 우클릭 하셔서 이름 바꾸기로 .py를 추가해주시면 됩니다!




# 2. 파이썬 파일 코딩 후 실행해보기



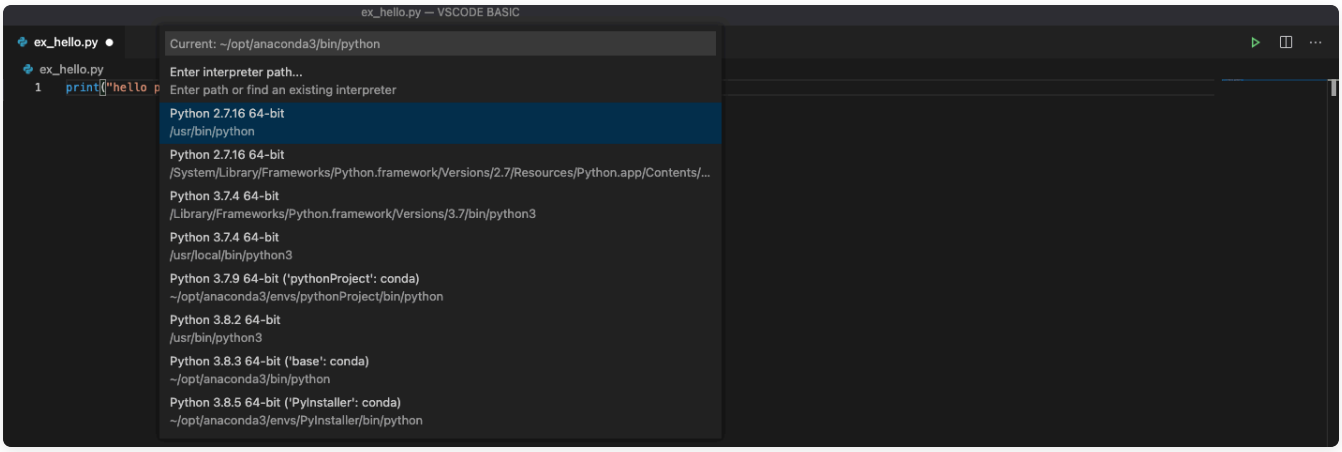
자! 이제 드디어 파이썬 언어로 코딩을 한 줄 해봅시다.

```
print("hello python")
```

아주 직관적인 파이썬 답게 "hello python"을 출력해 달라는 내용입니다.

우측 상단의 초록색 재생 버튼을 눌러서, 코드를 실행시키기 전에!

만약 아나콘다를 설치하셨다면 좌측 하단에 Python 3.8.3 64-bit('base':conda)라고 나와 있는 것을 보실 수 있을텐데요. 이것은 아나콘다를 설치하면 기본으로 구축되는 인터프리터 환경(environment)입니다. 나중에 배우시겠지만 conda install을 이용해서 라이브러리를 설치하면 여기 'base'에 라이브러리들이 설치되는데, 만약 다른 환경을 선택하신다면 잘 작동하던 코드가 오류가 뜰 수 있으니 신경써 주셔야해요.



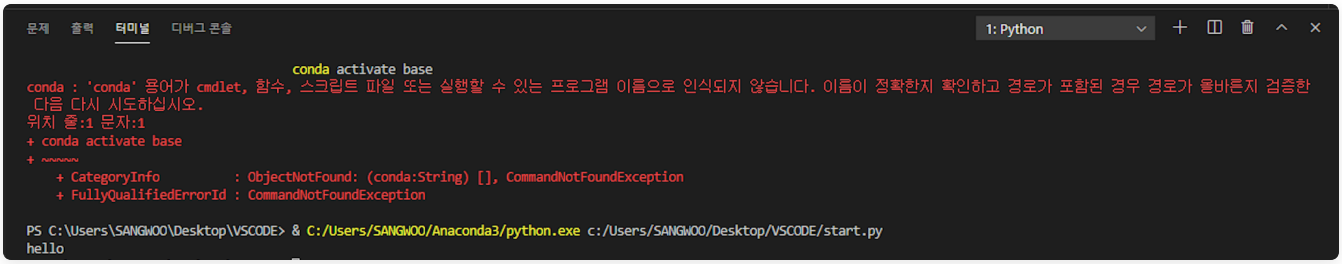

만약 python을 따로 설치하셨거나, 별도로 가상 환경을 생성하신 분들은 좌측 하단의 버전을 클릭하시면 위의 그림처럼 여러가지 환경을 선택하실 수 있는 창이 나타날 거에요.

# 3. 오류 해결 (기본 터미널 설정)

---

다시 본론으로 돌아와서 우측 상단의 코드 실행 버튼 (초록색 재생 버튼)을 누르시면 아마도 하단에 아래와 같은 글씨들이 뜰 거에요.

(Window OS는 이런 현상이 나타났고, Mac OS는 설치한지가 오래되어서 뜨는지 안뜨는지 잘 모르겠습니다.)

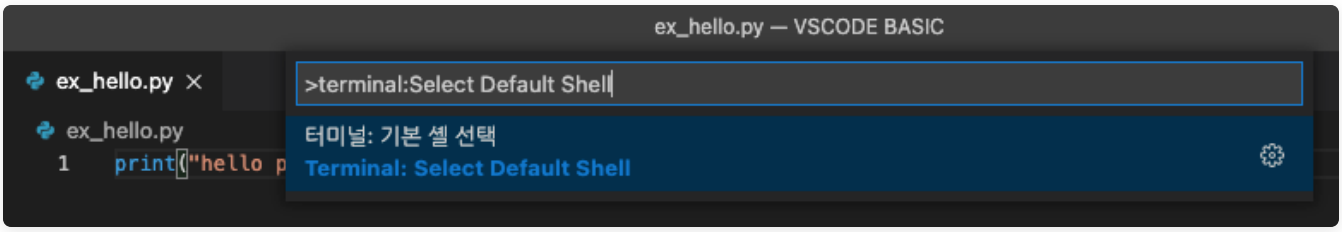


실행이 되긴 하는데 계속 빨간 글씨가 뜨는게 마음에 안듭니다.

많은 시행 착오 끝에 알아낸 원인은 VS code 설치 시, 기본 터미널이 powershell이라는 것입니다.

해결 방법은 아래와 같습니다.

1) Ctrl + Shift + P를 눌러서 Command Pallete를 켠 후, Terminal: Select Default Shell을 입력하여 선택




2) 항목들 중, Command Prompt를 선택 3) Terminal 재실행

# 4. 주의할 점

구글에 'vscode anaconda 초기설정'이라고 검색하면 나오는 블로그 중 한 곳에서, Code Runner Extension을 설치하라고 나옵니다.

저의 경우엔 이걸 설치하고 나니깐 코드 실행할 때마다 터미널에 실행 결과가 나오는 것이 아니라, 아래처럼 출력 탭을 통해 이렇게 나왔습니다.

만약 Code Runner Extension을 설치하셨다면 일단 다시 제거하시는게 좋을 것 같습니다.



# 2-5. VS CODE 사용법 (디버깅)

오늘은 열심히 프로그래밍의 영원한 동반자인 디버깅(Debugging)에 대해서 설명드리고 VS code에서 어떻게 효율적인 디버깅을 할 수 있는지 함께 알아보는 시간을 가지겠습니다.

# 1. 디버그(debug)란?

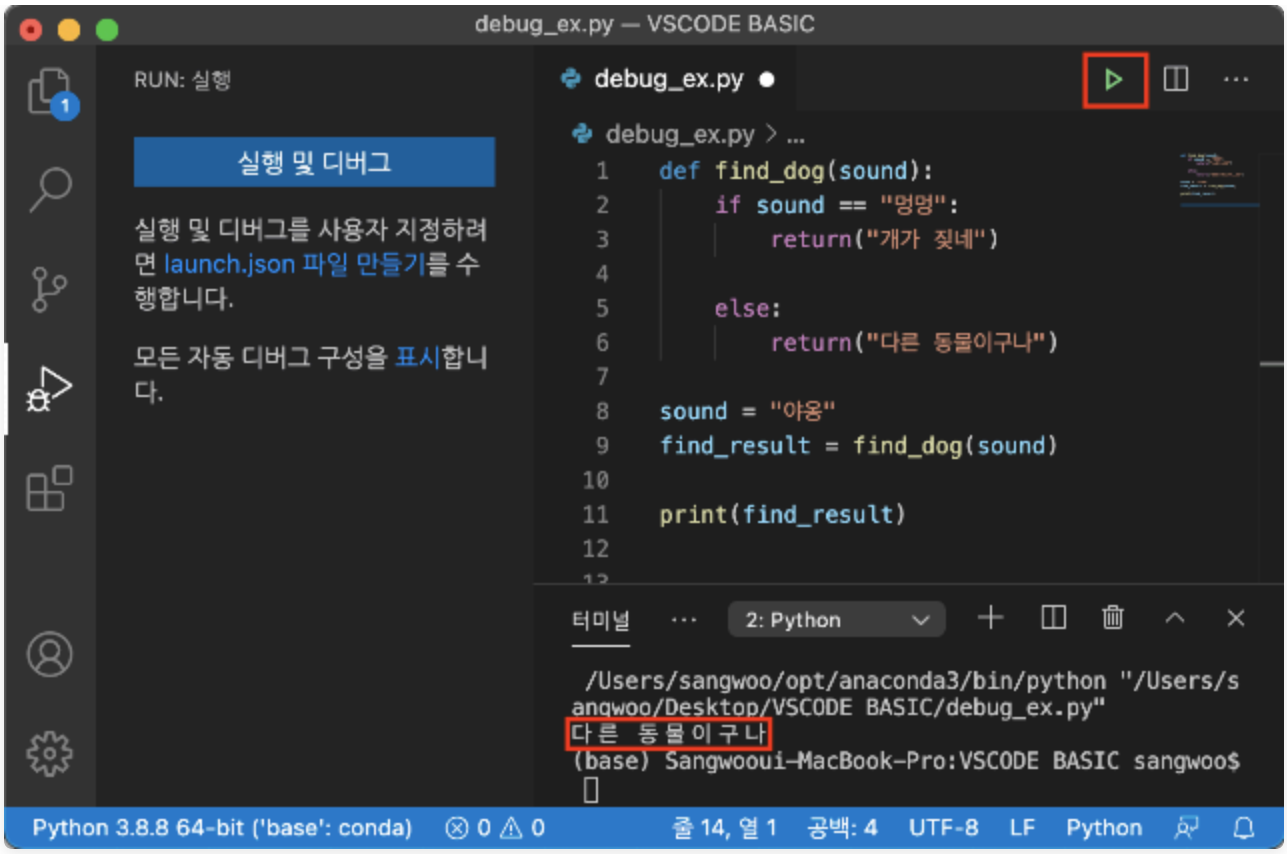
디버그는 "벌레를 잡다"라는 뜻으로 프로그램에서 의도치 않게 실행되는 오류를 수정하는 작업을 뜻합니다.

단번에 생각한 것을 완벽하게 코딩하는 것은 어려운 일입니다. 우리같은 초보에게는 짧은 몇 줄의 코드에서 조차 빈번하게 오류가 발생하죠.

물론 더 실력이 올라가면 그 수준에 맞는 어렵고 복잡한 코드를 작성해야하기에 디버깅은 개발자의 영원한 숙제라고도 할 수 있겠습니다.

우리는 지금까지 어떤 변수 안에 있는 값을 확인 하기 위해서는 print( ) 함수를 이용해서 터미널 창에서 확인해왔습니다.

이렇게 간단한 코드는 처음부터 끝까지 실행시키는 방식으로 가능하지만, 만약 수백, 수천 줄의 코드는 그런 방법은 너무 비효율 적이겠죠?




코드 실행 후, 터미널의 출력 창 확인

# 2. VS CODE 디버깅을 위한 설정

간단한 코드를 통해 VS CODE에서 어떻게 디버깅할 수 있는지 알아보겠습니다. 먼저 VS CODE를 켜서 아래의 코드를 복붙해 주세요.

개소리("멍멍")를 입력받으면 "개가 짖네"라는 문장을 출력하고, 그 외의 경우는 "다른 동물이구나"라는 문장을 출력하는 코드입니다.

```
def find_dog(sound):
    if sound == "멍멍":
        return("개가 짖네")

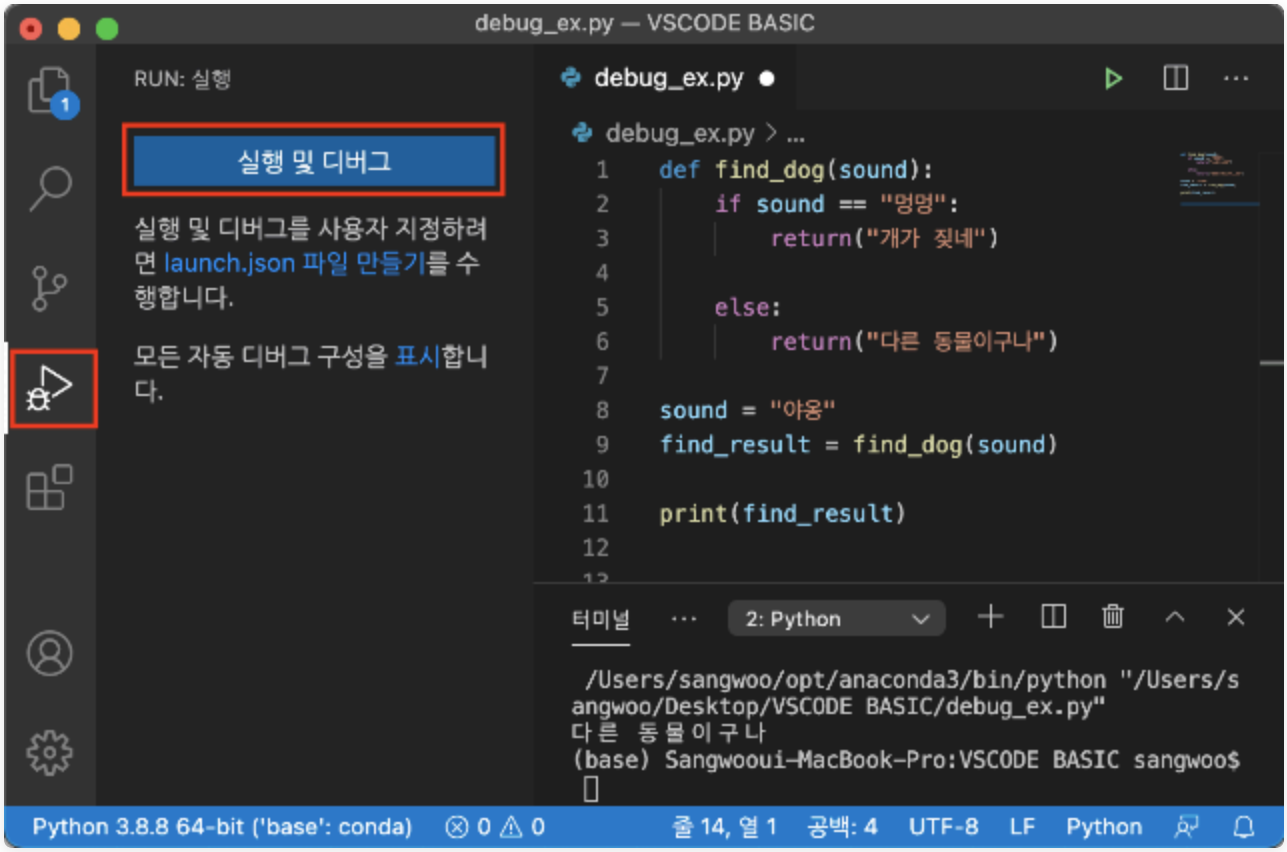
    else:
        return("다른 동물이구나")

sound = "야옹"
find_result = find_dog(sound)

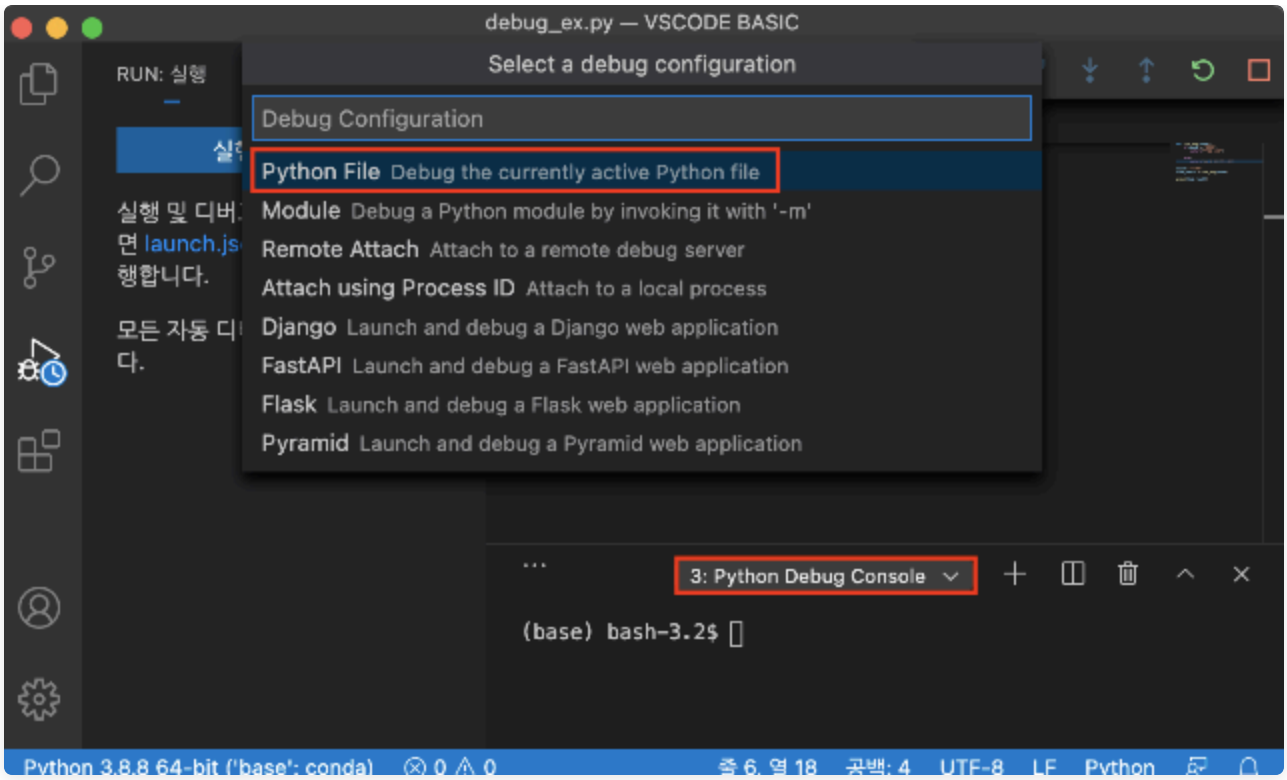
print(find_result)
```

코드를 입력하신 후, 화면 왼쪽에 벌레와 재생버튼을 합쳐놓은 듯한 아이콘을 클릭해주세요.

그러면 파란색으로 **'실행 및 디버그'** 버튼이 나오는데 이걸 클릭해주세요.




버튼을 클릭하시면 아래와 같이 화면 상단에 선택창이 뜨는데 Python File을 눌러주시면 됩니다. 그러면 잠시 하단의 파란색 상태바가 붉은색으로 바뀌었다가 원래 상태로 돌아오면서 터미널 창에 코드 실행결과가 출력되는 것을 확인 하실 수 있을텐데요. 사실 이렇게만 하면 우측 상단의 초록색 실행 버튼을 누른 것과 다른 것이 없습니다.



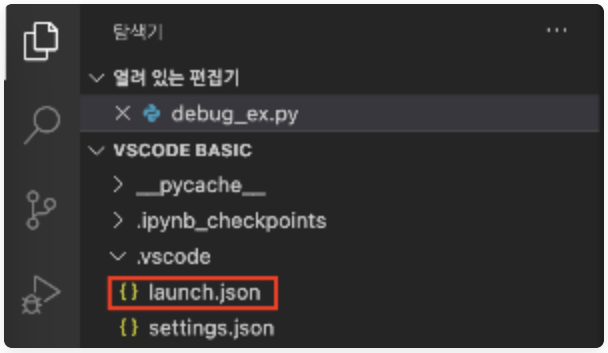

---

**<여기서 잠깐>**

항상 이렇게 Python File을 선택하는 번거로움을 없애기 위해 **아래와 같이 설정하시는 것을 추천**드립니다.

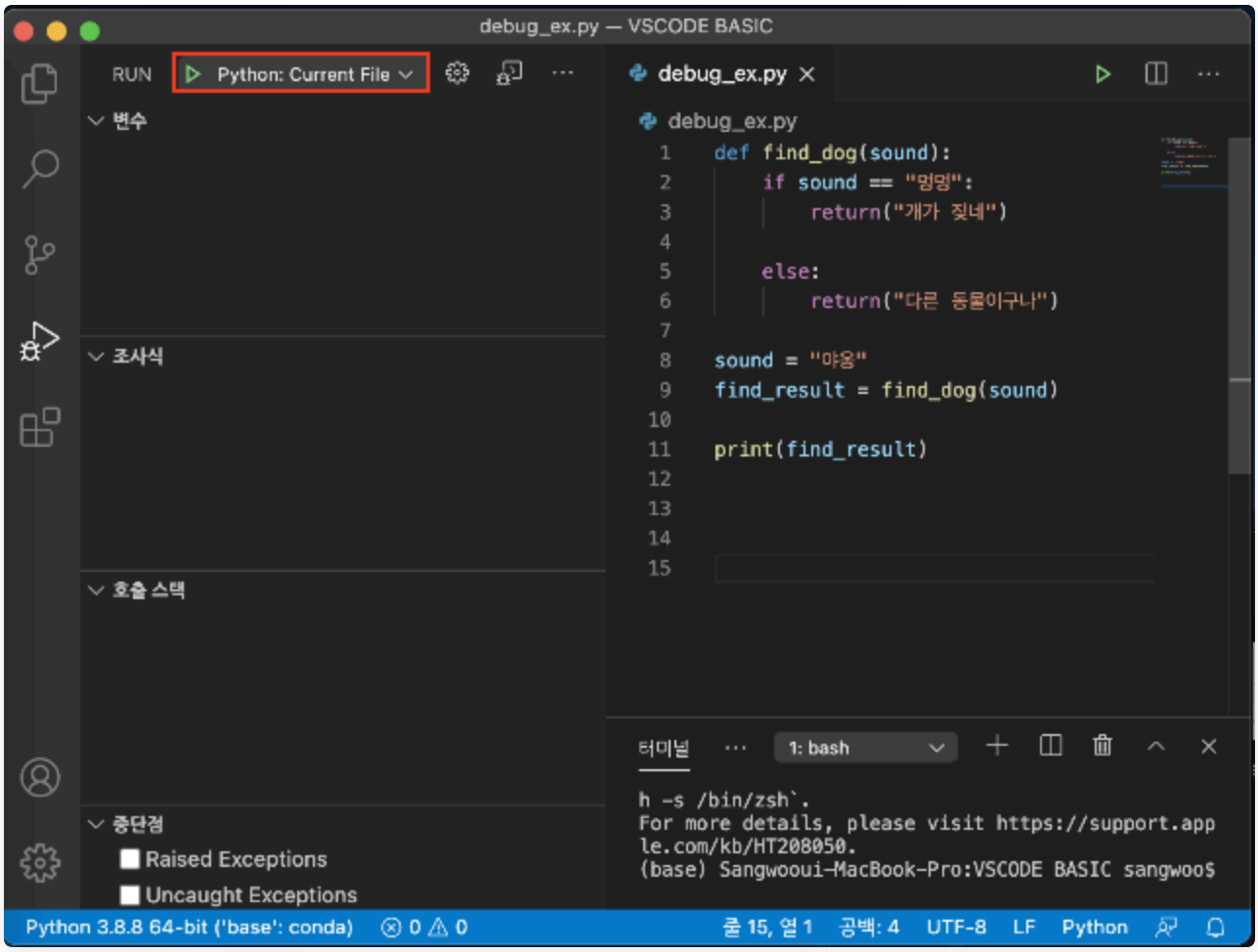
- 실행 및 디버그 버튼 밑에 **launch.json 파일 만들기** 를 클릭한 후, 위와 동일하게 Python File을 선택.
- launch.json 파일이 새로운 탭으로 뜨면 그냥 종료. (해당 프로젝트 폴더 내의 .vscode 안에 저장되어 있음)

(단, 이 설정은 각 프로젝트 폴더에서만 유효하며, 다른 프로젝트 폴더를 생성 시 다시 설정이 필요함.)




설정이 완료되면 아까의 **실행 및 디버그 버튼**이 없어지고 **초록색 Run 버튼**과 Python: Current File 이 생긴 것을 보실 수 있습니다.

아래에 변수(VARIABLES), 조사식(WATCH), 호출 스택(CALL STACK)이라는 창이 3개도 추가되었는데 이건 아래에서 설명드리겠습니다.




여기까지 따라 오셨으면 이제 디버깅에 필요한 셋팅이 모두 끝났습니다.

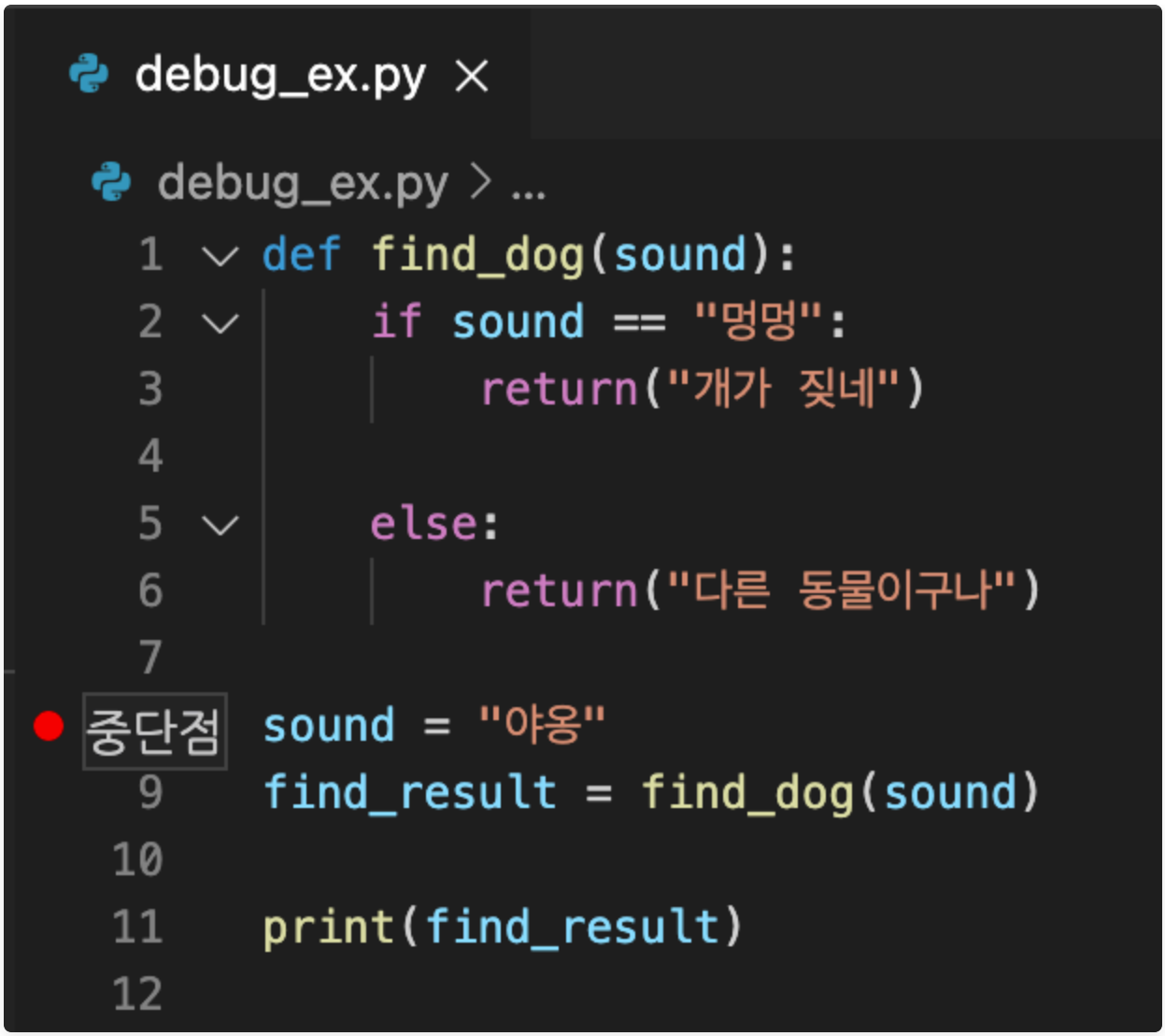
지금 부터는 어떻게 디버깅을 할 수 있는지 함께 알아보겠습니다.

---

# 3. VS CODE 디버깅

### 1) 중단점(Break Point) 지정하기

코드의 라인넘버 왼쪽의 공간에 커서를 올리면 Break Point를 찍을 수 있습니다. 중단점을 찍는 것은 해당 라인 이전까지만 실행시키기 위함입니다. **즉, 빨간색 점으로 표시한 라인 전까지 코드가 실행되기를 원한다는 의미입니다.**




### 2) Run 버튼 클릭(디버깅 실행)

이번엔 가장 첫 라인에 중단점을 찍고 좌측 상단의 Run 버튼을 눌러보겠습니다. (우측 상단이 아님에 주의)

아래와 같은 화면이 출력됩니다. 현재는 첫 줄의 코도도 실행되지 않은 상태입니다. 여기서 다음 코드를 한 줄씩 실행시켜주려면 빨간색으로 표시한 아이콘을 클릭해주시면 됩니다.

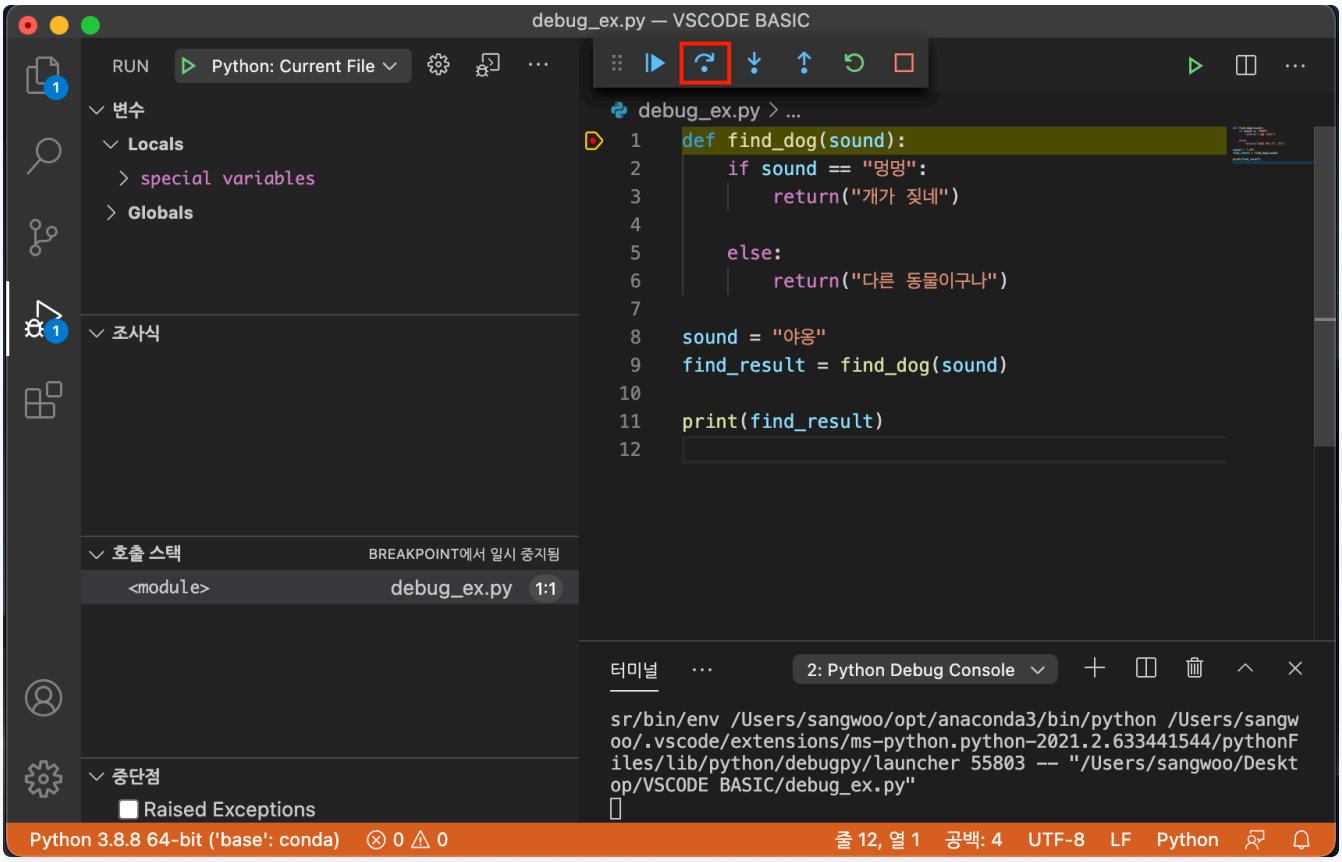
여러분들도 눌러보시면서 **코드 실행 순서가 어떻게 흘러가는지, 왼쪽의 변수창과 호출 스텍창이 어떻게 변하는지 관찰**해보세요.

8번 줄의 코드를 실행시키고 나면 sound: '야옹'이라는 변수가 추가되는 것도 확인해보시고, 가장 마지막 11번 줄 실행 직전에는 어떤 변수들이 생겼는지도 확인해보시면 좋을 것 같아요.

**<여기서 잠깐>**

**- 변수(variables) 창에 나타나있는 변수를 더블클릭하면 값을 변경 가능. 다음 코드부터 변경된 값으로 진행**

**- 함수(def) 부분이 실행될 때는 호출 스택을 통해서 어디를 통해서 함수가 호출되었는지도 확인 가능**



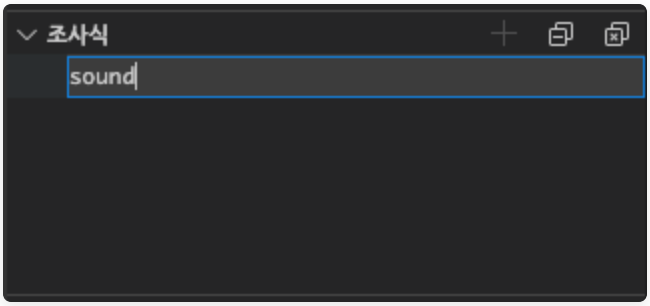

## 3) 조사식(WATCH) 사용법

위의 1, 2번 단계만 잘 사용하셔도 충분하지만, 조사식 사용법도 알아두시면 좋을 것 같아요.

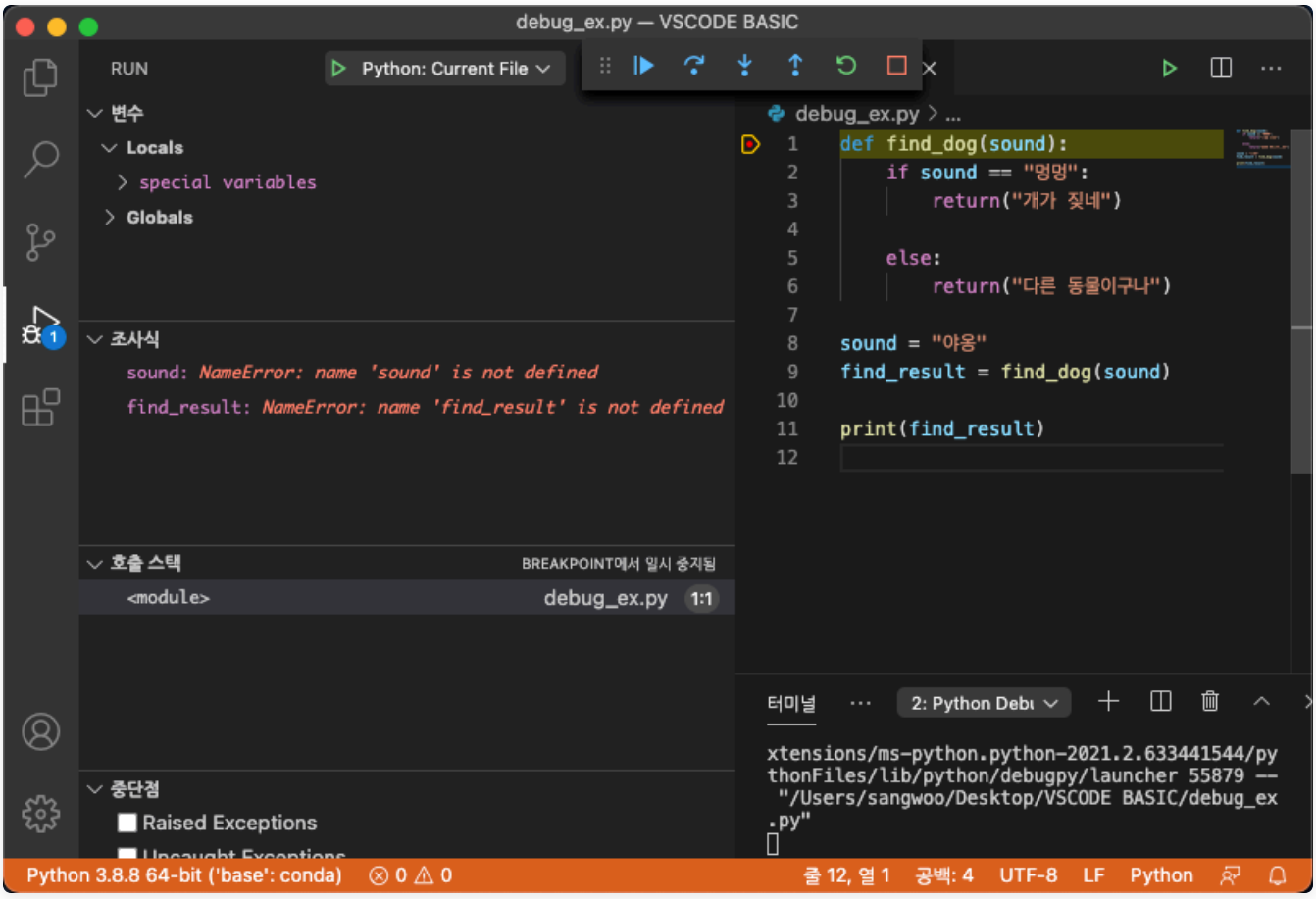
조사식 창에 커서를 올리시면 아래와 같이 + 아이콘이 나타납니다. 이걸 클릭해주세요.




클릭하면 입력창이 뜨는데, 여기에 확인하고 싶은 변수명을 입력해줍니다.

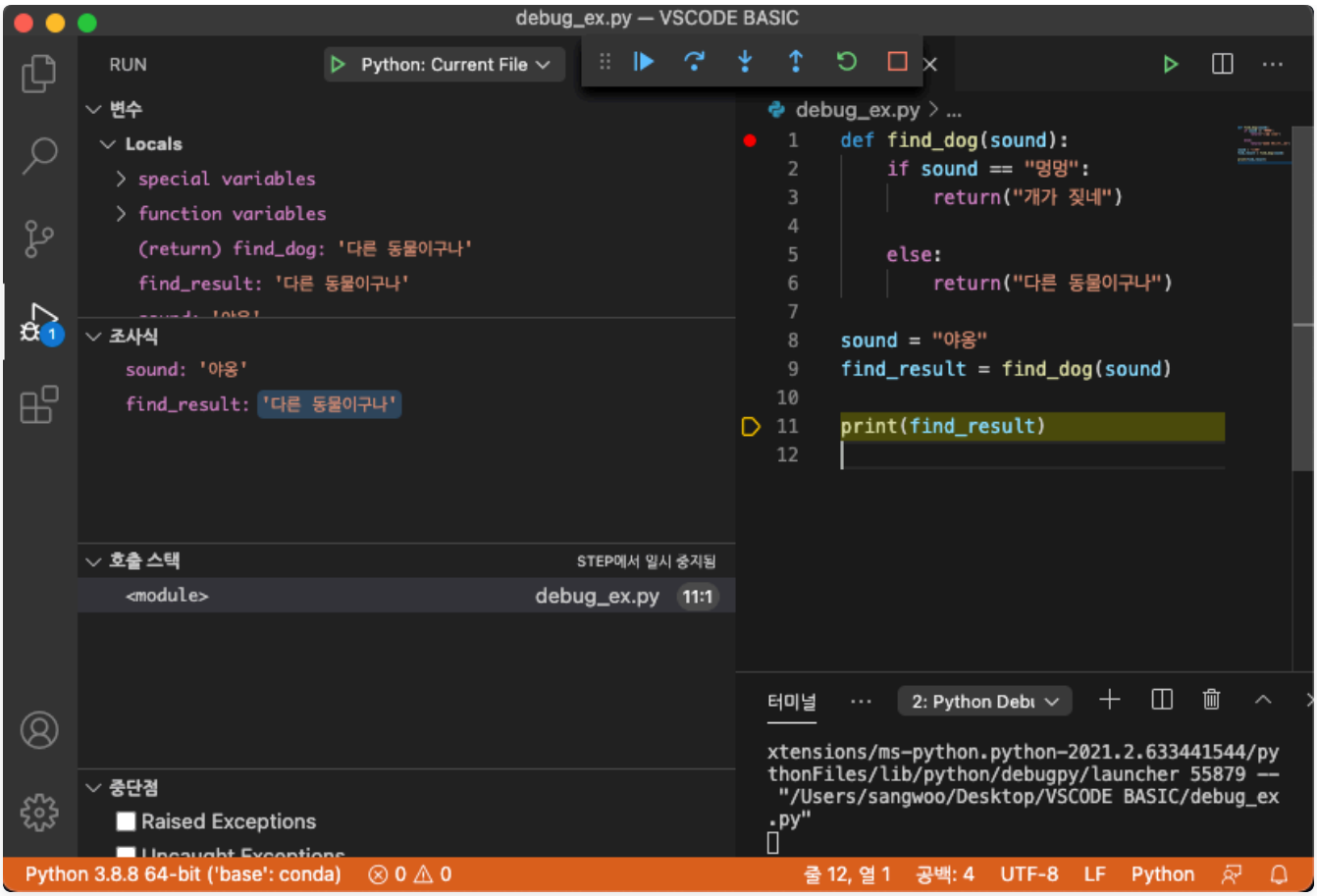


저는 sound와 find_result를 입력해보았습니다. 현재는 디버깅을 시작했지만 아무것도 실행되지 않은 상태여서 해당 변수들이 정의되지 않은 상태라며 에러를 출력하고 있습니다.




쭉쭉 한 줄씩 실행시켜보면 코드가 진행됨에 따라 변수에 값이 들어가있는 것을 확인 하실 수 있습니다.

이 기능은 내가 관찰하고 싶은 변수가 어떻게 변하는지 집중해서 볼 때 유용할 것 같아요.


debug_ex.py — VSCODE BASIC
RUN
Python: Current File
변수
Locals
special variables
function variables
(return) find_dog: '다른 동물이구나'
find_result: '다른 동물이구나'
조사식
sound: '아웅'
find_result: '다른 동물이구나'
호출 스택
STEP에서 일시 중지됨
<module>
debug_ex.py
11:1
중단점
Raised Exceptions
debug_ex.py > ...
1 def find_dog(sound):
2 if sound == "멍멍":
3 return("개가 짖네")
4
5 else:
6 return("다른 동물이구나")
7
8 sound = "아웅"
9 find_result = find_dog(sound)
10
11 print(find_result)
12
터미널
2: Python Debu
xtensions/ms-python.python-2021.2.633441544/pythonFiles/lib/python/debugpy/launcher 55879 -- "/Users/sangwoo/Desktop/VSCODE BASIC/debug_ex.py"
Python 3.8.8 64-bit ('base': conda)
0 0
줄 12, 열 1
공백: 4
UTF-8
LF
Python